

Basics of Image Processing

Carsten Rother

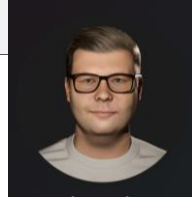
04/11/2025



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386



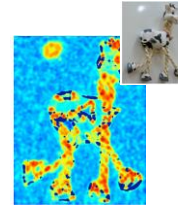
Lecture Content



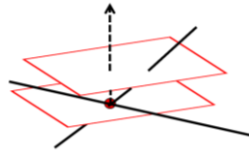
- Introduction (~1.5h):
- Introduction to Deep Machine Learning (~2h):



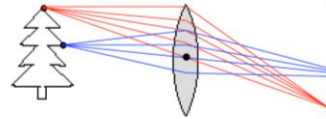
- **Basics of Image Processing (~2.5h)**



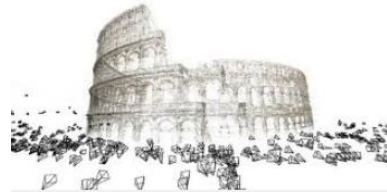
- Basics of Projective Geometry (~1h)



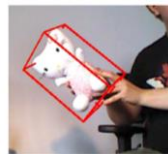
- What is a Camera? (~1h)



- 3D Reconstruction (~5h)



- Differentiable RANSAC (~1h)



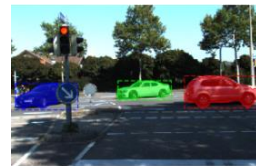
- Object Tracking (~1.5h)



- Object Recognition (~2h)



- Image Generation (~2h)



- Training Data Generation (~1.5h)



Roadmap: Basics of Digital Image Processing

- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

What is an Image

- We can think of an image as a function:

$$I(x, y), \quad I: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$$

- For every 2D point (pixel) it tells us the amount of light it receives

- The **size** and **range** of the sensor is limited:

$$I(x, y), \quad I: [a, b] \times [c, d] \rightarrow [0, m]$$

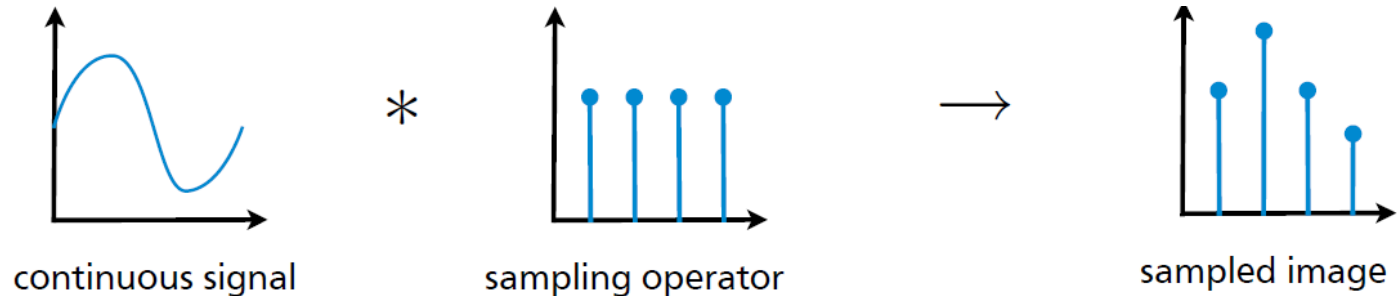
- **Colour image** is then a vector-valued function:

$$I(x, y) = \begin{pmatrix} I_R(x, y) \\ I_G(x, y) \\ I_B(x, y) \end{pmatrix}, \quad I: [a, b] \times [c, d] \rightarrow [0, m]^3$$

- Comment, in most lectures we deal with grey-valued images and extension to colour is often “obvious”

Digital Images

- We usually do not work with spatially continuous functions, since our cameras do not sense in this way
- Instead we use spatially discrete images
- Sample the 2D domain on a regular grid (1D version)



- Intensity/colour values are usually also discrete.
Quantize the values per channel
(e.g. 8 bit per channel)

		x =															
		58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	
y =	41	210	209	204	202	197	247	143	71	64	80	84	54	54	57	58	
	42	206	196	203	197	195	210	207	56	63	58	53	53	61	62	51	
	43	201	207	192	201	198	213	156	69	65	57	55	52	53	60	50	
	44	216	206	211	193	202	207	208	57	69	60	55	77	49	62	61	
	45	221	206	211	194	196	197	220	56	63	60	55	46	97	58	106	
	46	209	214	224	199	194	193	204	173	64	60	59	51	62	56	48	
	47	204	212	213	208	191	190	191	214	60	62	66	76	51	49	55	
	48	214	215	215	207	208	180	172	188	69	72	55	49	56	52	56	
	49	209	205	214	205	204	196	187	196	86	62	66	87	57	60	48	
	50	208	209	205	203	202	186	174	185	149	71	63	55	55	45	56	
	51	207	210	211	199	217	194	183	177	209	90	62	64	52	93	52	
	52	208	205	209	209	197	194	183	187	187	239	58	68	61	51	56	
	53	204	206	203	209	195	203	188	185	183	221	75	61	58	60	60	
	54	200	203	199	236	188	197	183	190	183	196	122	63	58	64	66	
	55	205	210	202	203	199	197	196	181	173	186	105	62	57	64	63	

Roadmap: Basics of Digital Image Processing

- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

Point operators

- Point operators work on every pixel independently:

$$J(x, y) = h(I(x, y))$$

- Examples for h :

- Control contrast and brightness: $h(z) = az^b + c$



original



Contrast enhanced

Roadmap: Basics of Digital Image Processing

- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

Linear Filters / Operators

- Properties of a **linear operator** T :
 - Homogeneity: $T[aX] = aT[X]$
 - Additivity: $T[X + Y] = T[X] + T[Y]$
 - Superposition: $T[aX + bY] = aT[X] + bT[Y]$
- Example:
 - Convolution
 - Matrix-Vector operations

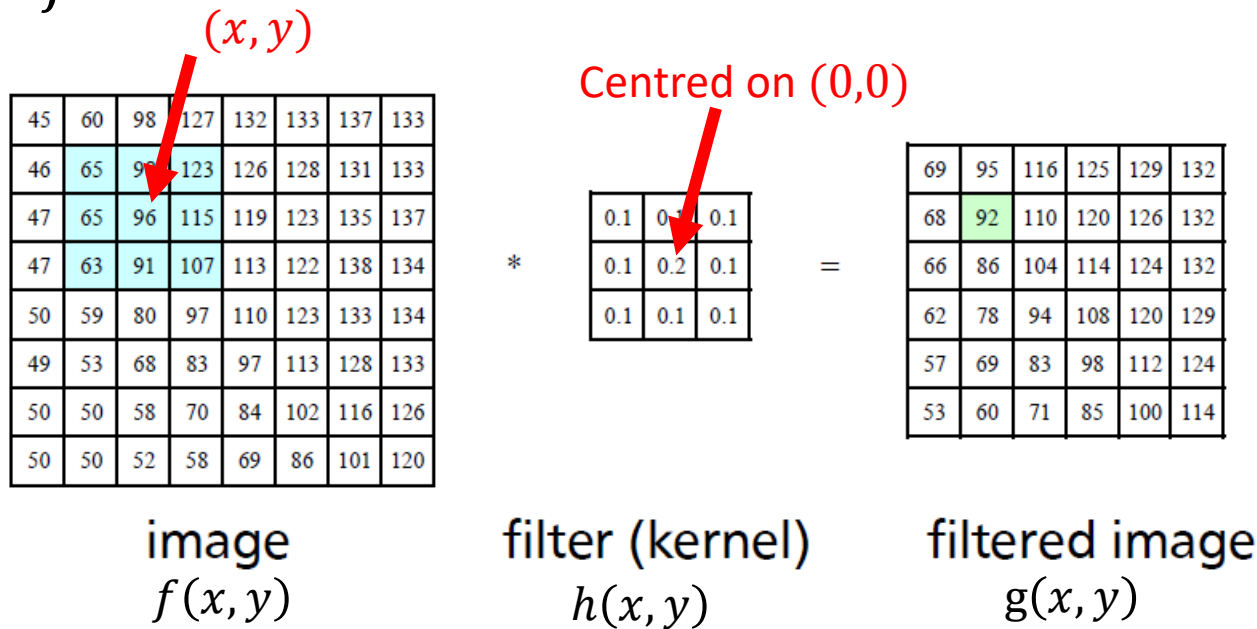
Convolution

- Replace each pixel by a linear combination of its neighbours and itself

Convolution

- Replace each pixel by a linear combination of its neighbours and itself
- 2D convolution (discrete)

$$g = f * h$$



$$g(x, y) = \sum_{k, l} f(x - k, y - l) h(k, l)$$

“the image f is implicitly mirrored if $k = -1 \dots 1$ ”

Convolution - Properties

- Superposition, i.e. $h * (af_0 + bf_1) = ah * f_0 + bh * f_1$
- Associative $(f * g) * h = f * (g * h)$
- Commutative $f * h = h * f$
- Can be written in Matrix form: $g = H f$
- Correlation (not mirrored filter):

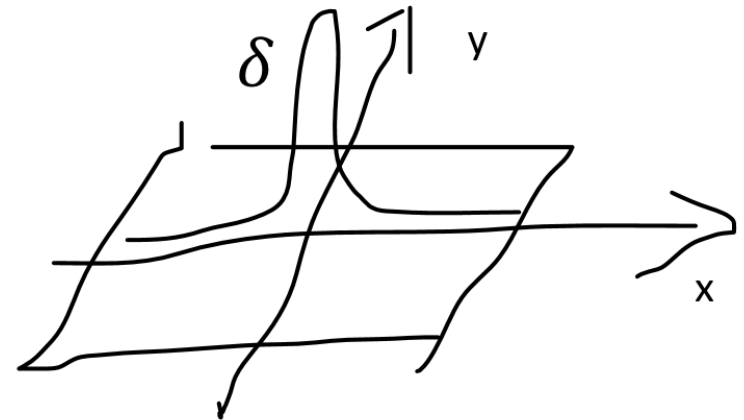
$$g(x, y) = \sum_{k,l} f(x + k, y + l)h(k, l)$$

$$\begin{bmatrix} 72 & 88 & 62 & 52 & 37 \end{bmatrix} * \begin{bmatrix} 1/4 & 1/2 & 1/4 \end{bmatrix} \Leftrightarrow$$

$$\frac{1}{4} \begin{bmatrix} 2 & 1 & . & . & . \\ 1 & 2 & 1 & . & . \\ . & 1 & 2 & 1 & . \\ . & . & 1 & 2 & 1 \\ . & . & . & 1 & 2 \end{bmatrix} \begin{bmatrix} 72 \\ 88 \\ 62 \\ 52 \\ 37 \end{bmatrix}$$

Examples

- Impulse function: $f = f * \delta$



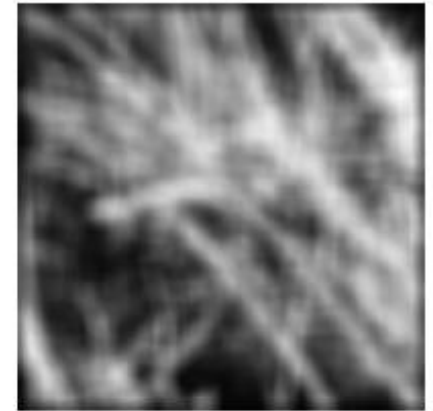
- Box Filter:

$$\frac{1}{9} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Original Image



Box-filtered image



Application: Noise removal

- Filtering can be used to remove noise
- Sources for noise: sensor noise (shot noise, camera-internal noise), quantisation artefacts, etc
- Typical assumption is that the noise of neighbouring pixels is uncorrelated
- **Basic Idea: neighbouring pixels contain information about the intensity of centre pixel**

2	3	3
3	20	2
3	2	3

 →

2	3	3
3	3	2
3	2	3

Box Filter for noise removal

- Box filter takes the mean in a neighbourhood



Image



Noise



Pixel-independent
Gaussian noise added

$$\frac{1}{9} \cdot \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$



Filtered Image

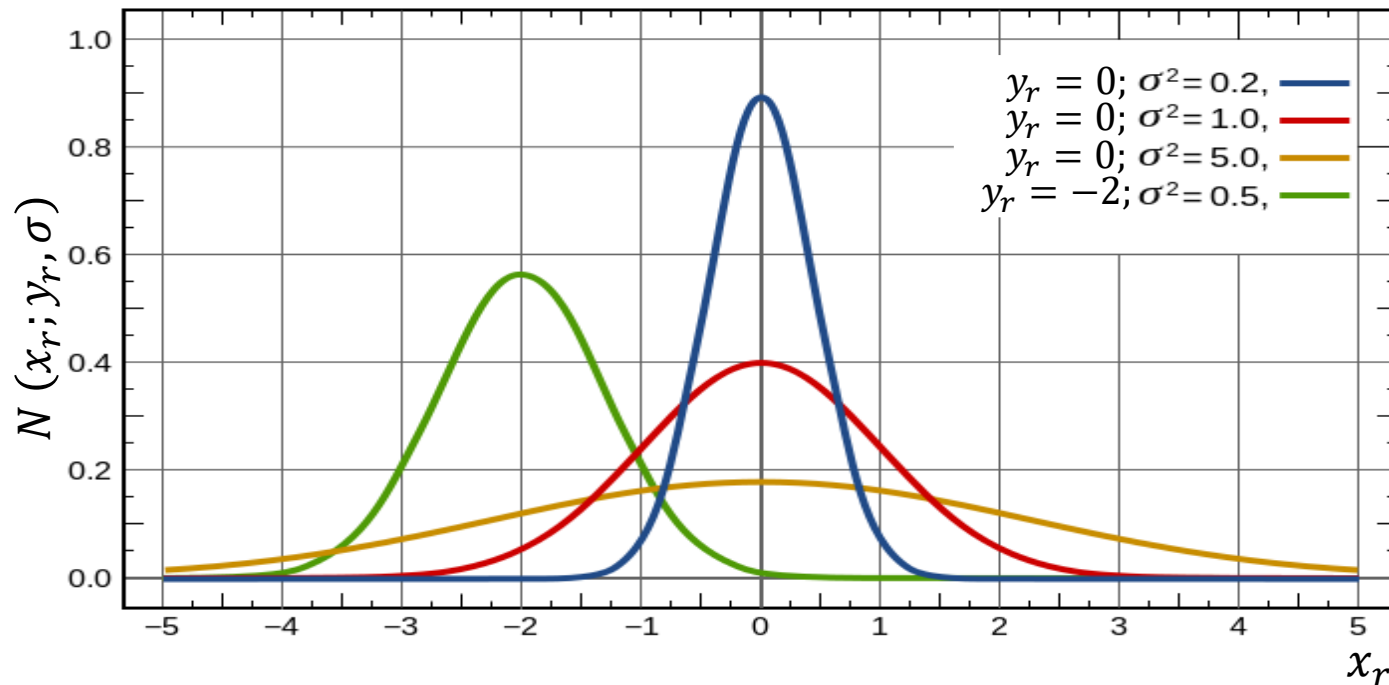
Derivation of Box Filter

- y_r is true gray value (color)
- x_r observed gray value (color)
- Noise model: Gaussian noise:

Means equal
up to scale

$\exp[x] = e^x$
($e = 2.71828 \dots$)

$$p(x_r | y_r) = N(x_r; y_r, \sigma) \sim \exp\left[-\frac{(x_r - y_r)^2}{2\sigma^2}\right]$$



Derivation of Box Filter

Derivation of Box Filter

- Further assumption: pixel-independent noise

$$p(x|y) \sim \prod_r \exp\left[-\frac{(x_r - y_r)^2}{2\sigma^2}\right]$$

- Find the most likely solution for the true signal y
Maximum-Likelihood principle (posterior maximization):

$$y^* = \underset{\text{posterior}}{\operatorname{argmax}_y} p(y|x) = \underset{\text{prior likelihood}}{\operatorname{argmax}_y} \frac{p(y)p(x|y)}{p(x)}$$

- $p(x)$ is a constant (drop it), assume (for now) uniform prior $p(y)$.

We get:

$$p(y|x) = p(x|y) \sim \prod_r \exp\left[-\frac{(x_r - y_r)^2}{2\sigma^2}\right]$$

➡ the solution is trivial: $y_r^* = x_r$ for all r ☹

➡ additional assumptions about the **signal y** are necessary

Derivation of Box Filter

Derivation of Box Filter

Assumption: not uniform prior $p(y)$ but ...

in a small vicinity $W(r) \subset D$ the “true” signal y_r is nearly constant

Maximum-a-posteriori:

Only one y_r in a window $W(r)$

$$p(y|x) \sim \prod_r \prod_{r' \in W(r)} \exp\left[-\frac{(x_{r'} - y_r)^2}{2\sigma^2}\right]$$

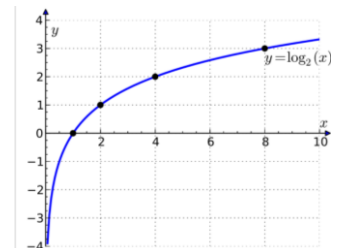
For one pixel r :

$$y_r^* = \operatorname{argmax}_{y_r} \prod_{r' \in W(r)} \exp\left[-\frac{(x_{r'} - y_r)^2}{2\sigma^2}\right]$$

take neg. logarithm ($\ln$):

$$y_r^* = \operatorname{argmin}_{y_r} \sum_{r' \in W(r)} \frac{(x_{r'} - y_r)^2}{2\sigma^2}$$

Note, if y^* is a maximizer of $f(y)$ then y^* is also of is a maximizer of $\log f(y)$ (since log-function is monotonically increasing, i.e. $x_1 \geq x_2$ means that $\log x_1 \geq \log x_2$)



Derivation of Box Filter

How to do the minimization:

$$y_r^* = \operatorname{argmin}_{y_r} 1/2\sigma^2 \sum (x_{r'} - y_r)^2$$

Take derivative and set to 0:

$$F(y_r) = 1/2\sigma^2 \sum (x_{r'} - y_r)^2$$

$$\frac{\partial F}{\partial y_r} = -1/\sigma^2 \left(\sum (x_{r'} - y_r) \right) = -1/\sigma^2 \left(\left(\sum x_{r'} \right) - |W|y_r \right) = 0$$

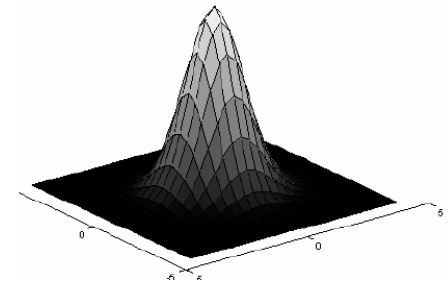
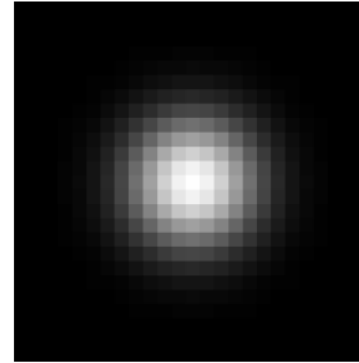
$$\Rightarrow y_r^* = \frac{1}{|W|} \sum_{r'} x_{r'} \quad (\text{the average})$$

Box filter is optimal under pixel-independent, Gaussian noise and constant signal in window

Gaussian (Smoothing) Filters

- Nearby pixels are weighted more than distant pixels
- Isotropic Gaussian (rotational symmetric)

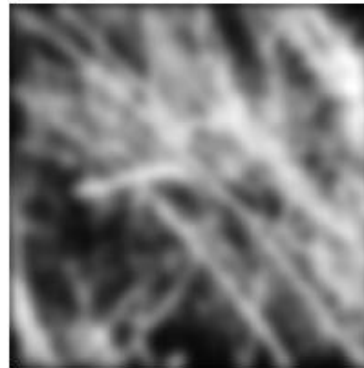
$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$



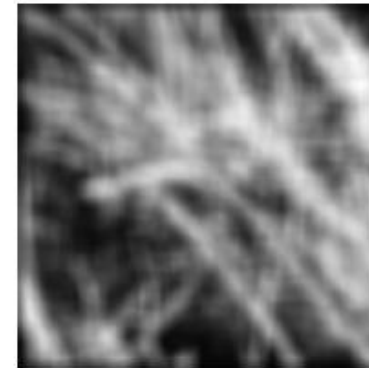
Original Image



Gaussian-filtered image

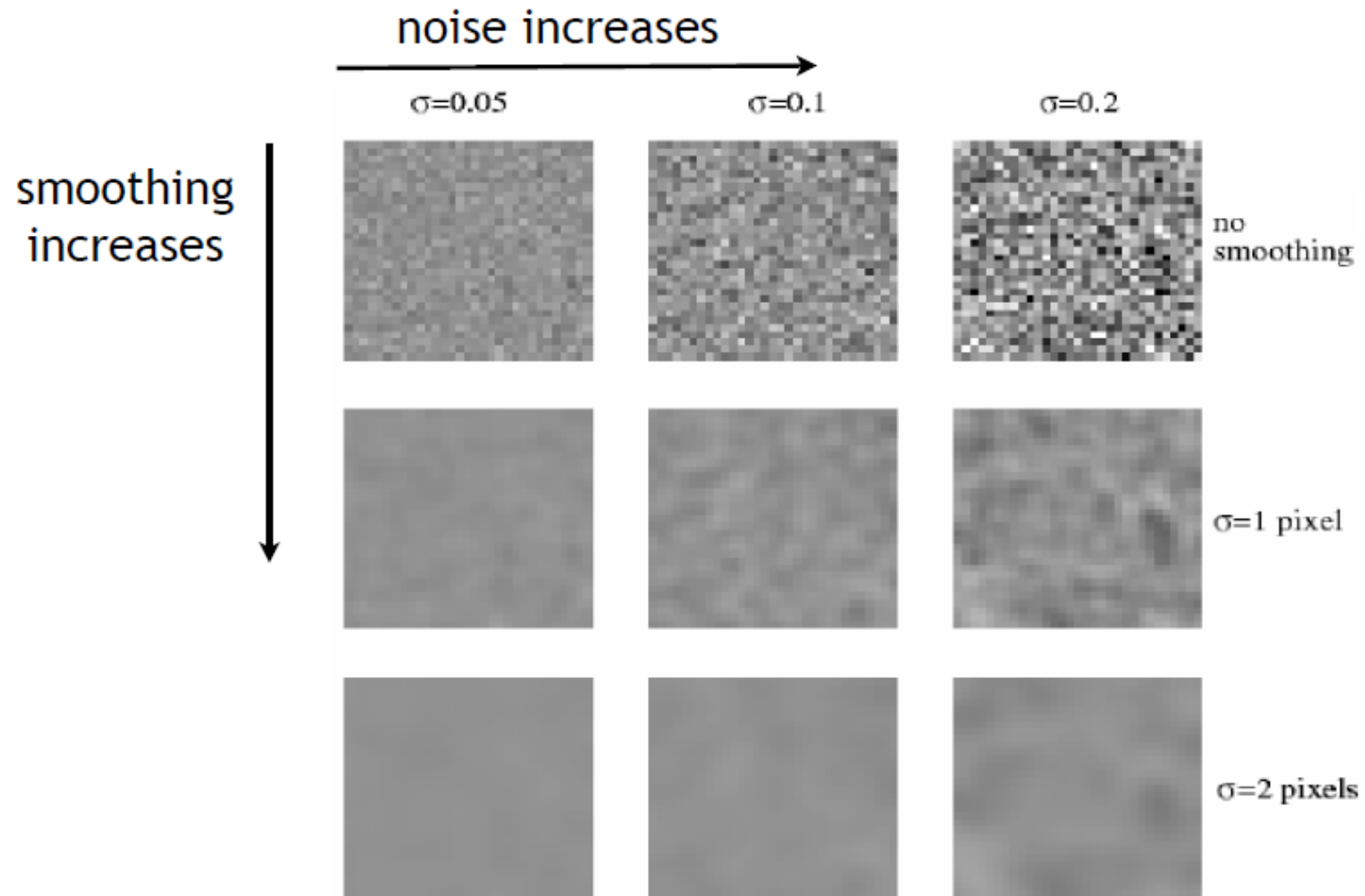


Box-filtered image



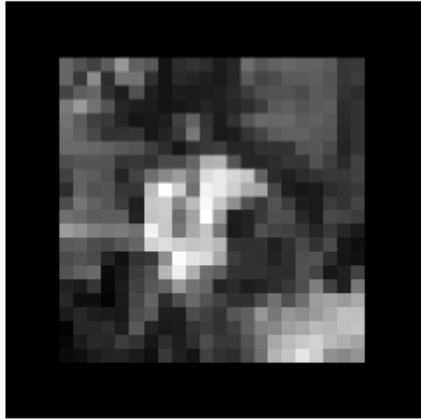
Gaussian Filter

Input: constant grey-value image

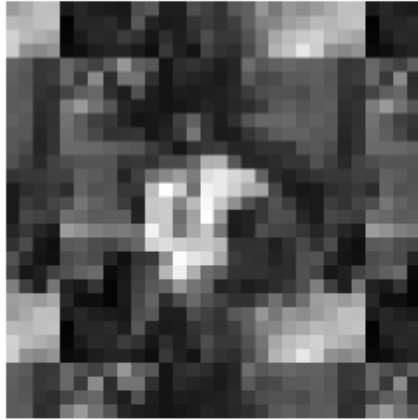


More noise needs larger sigma

Handling the Boundary (Padding)



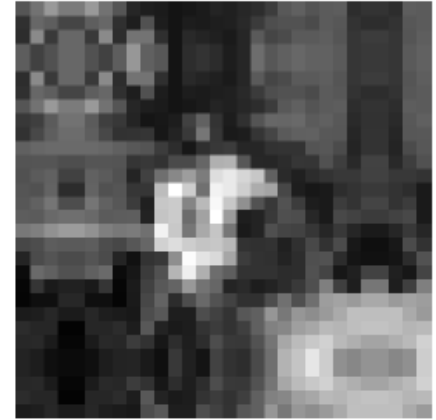
zero



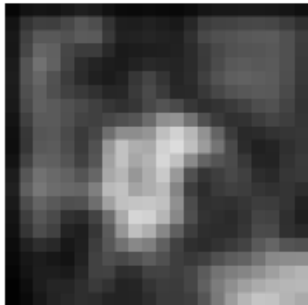
wrap



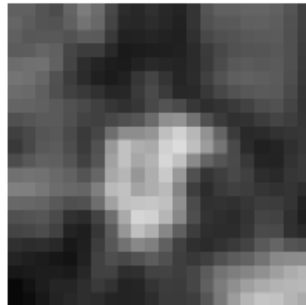
clamp



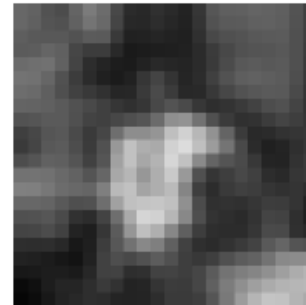
mirror



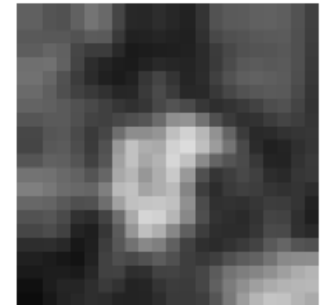
blurred zero



blurred warp

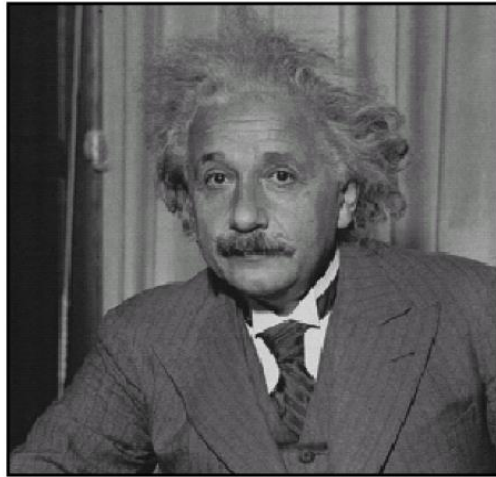


blurred clamp

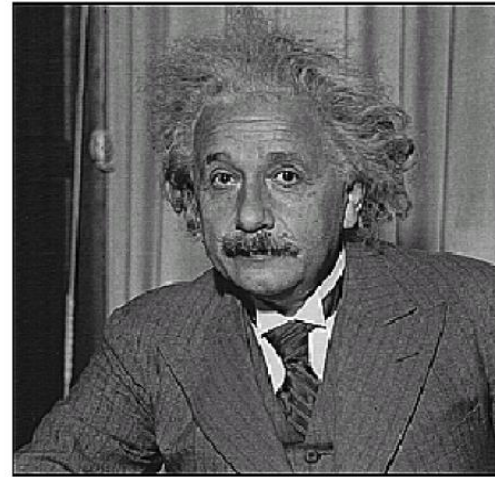


blurred mirror

Question



Original f



Output g

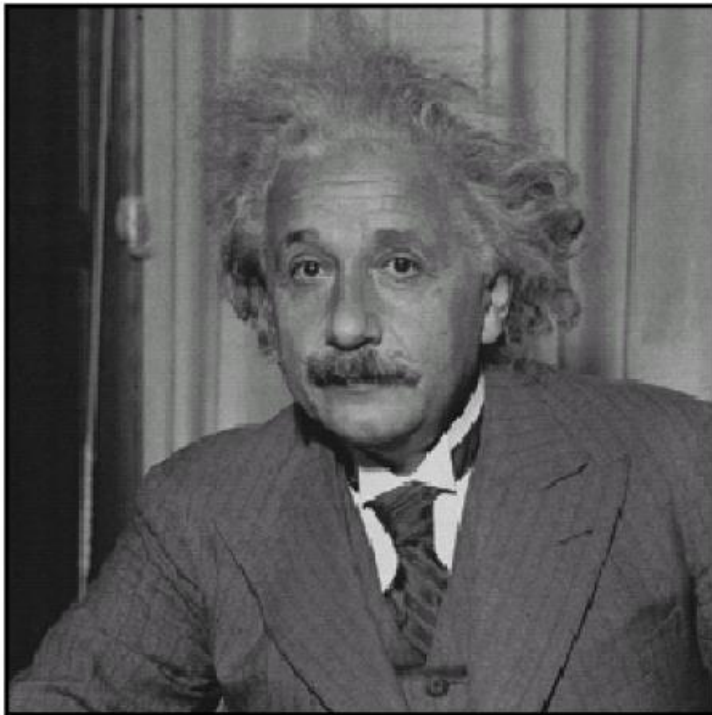
Which answer is correct:

- 1) Given f , it is not possible to obtain the sharp output g
- 2) $g = f + 0.5 (f - (h_{blur} * f))$
- 3) $g = f + 0.5 (f + (h_{blur} * f))$
- 4) I don't know

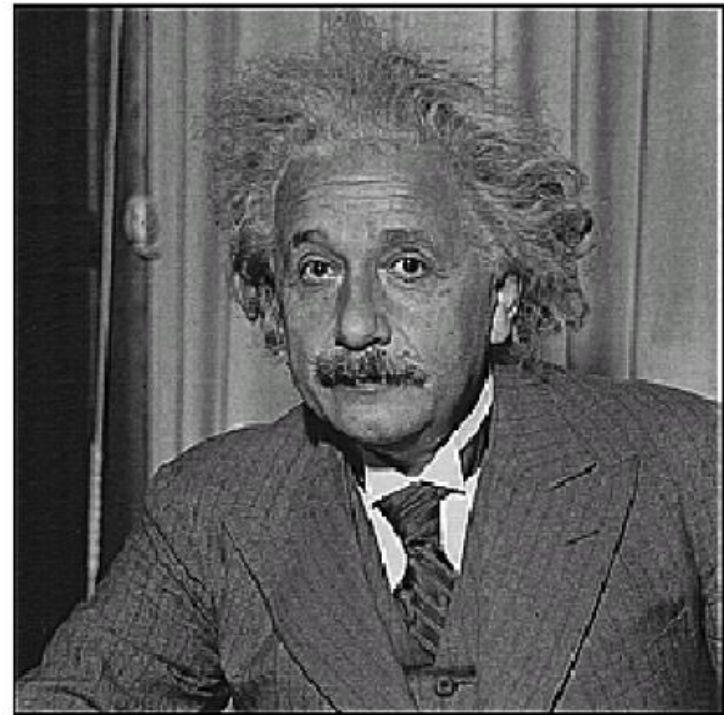
Gaussian for Sharpening

Sharpen an image by amplifying what “smoothing has removed”:

$$g = f + \gamma (f - (h_{blur} * f))$$



Original f

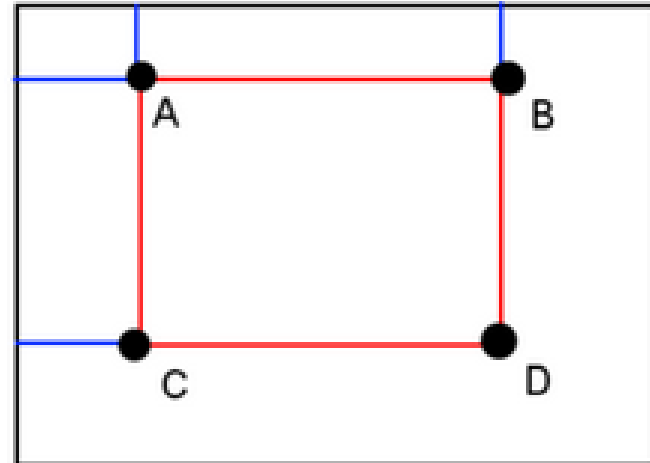


Output g

How to compute convolutions efficiently?

- Separable filters (next slides)
- Integral Image (Summed-area table) trick
 - Naive implementation has complexity $O(Nw)$ where N is number of pixels, and w is number of elements in a filter
 - The Box filter (and a few others) can be computed in $O(N)$ (see https://en.wikipedia.org/wiki/Summed-area_table)

- 1) Build a sum-table $O(N)$
- 2) Sum of Red square is $O(1)$:



$$\text{Sum} = D - B - C + A$$

- Fourier transformation

How it is done in Neural Networks

- Explicit computation
 - complicated with special cases, e.g. padding

- Fourier transformation
 - complicated with special cases, e.g. padding

$$\begin{bmatrix} B_{11} & B_{12} & 0 & \cdots & \cdots & 0 \\ B_{21} & B_{22} & B_{23} & \ddots & \ddots & \vdots \\ 0 & B_{32} & B_{33} & B_{34} & \ddots & \vdots \\ \vdots & \ddots & B_{43} & B_{44} & B_{45} & 0 \\ \vdots & \ddots & \ddots & B_{54} & B_{55} & B_{56} \\ 0 & \cdots & \cdots & 0 & B_{65} & B_{66} \end{bmatrix}$$

- **Best:** Write as explicit Matrix-Vector product: $B * I = R$
 - B is a band-matrix with kernel values, I is vector with image entries, and R is the resulting image vector
 - Well suited for graphics card (GPU)
 - Final solution: various small tricks, see article below

[Arxiv paper 2014: cuDNN: Efficient Primitives for Deep Learning, Section 3.1]

Separable filters

For some filters we have: $f * h = f * (h_x * h_y)$

Where h_x, h_y are 1D filters.

Example Box filter:

$$\frac{1}{9} \cdot \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} = \frac{1}{3} \cdot \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline \end{array} * \frac{1}{3} \cdot \begin{array}{|c|} \hline 1 \\ \hline 1 \\ \hline 1 \\ \hline \end{array}$$

$h_x * h_y$ h_x h_y

Now we can do two 1D convolutions (since associative):

$$f * h = f * (h_x * h_y) = (f * h_x) * h_y$$

Naïve implementation for 3x3 filter: 9N operations versus 3N+3N operations using separable filters

Can any filter be made separable?

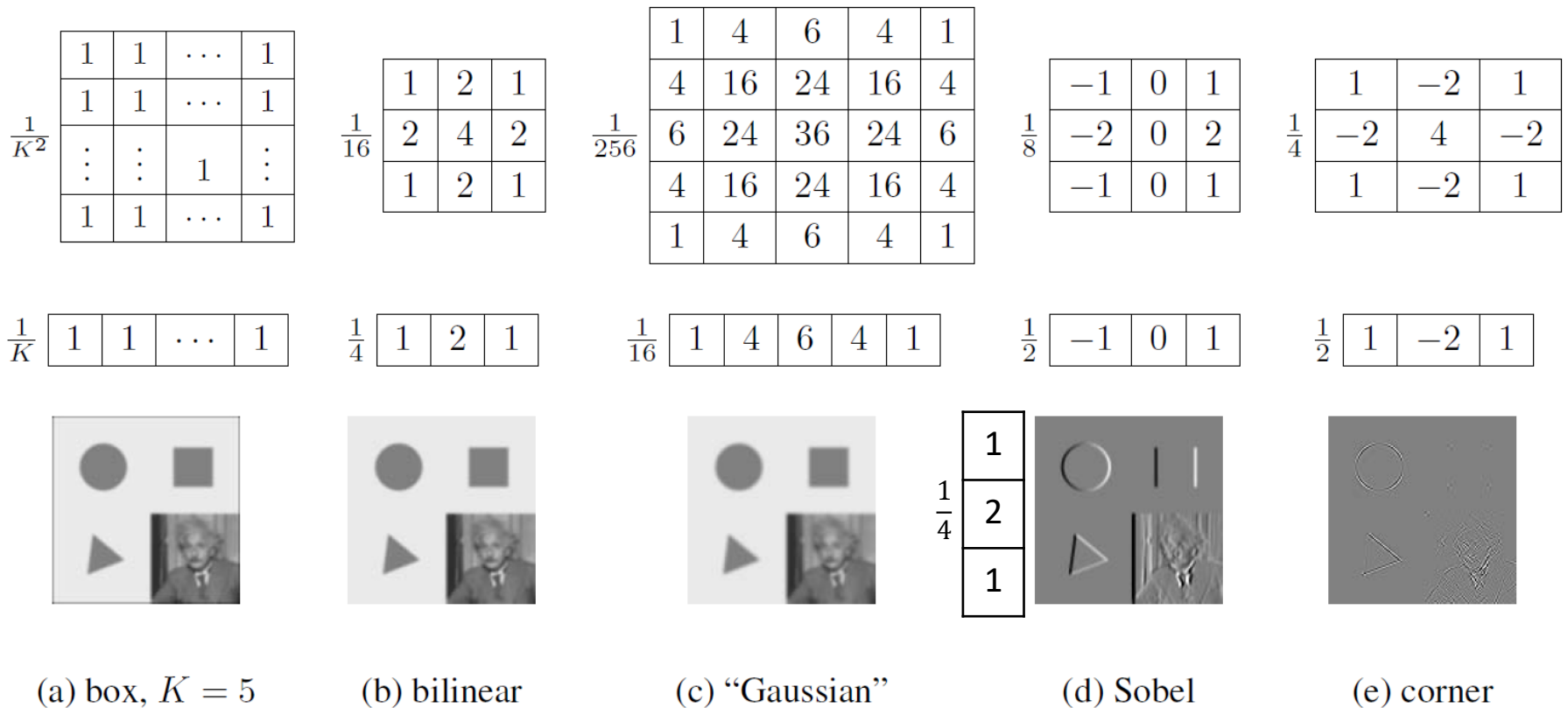
Note:
$$\frac{1}{9} \cdot \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} = \frac{1}{3} \cdot \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline \end{array} * \frac{1}{3} \cdot \begin{array}{|c|} \hline 1 \\ \hline 1 \\ \hline 1 \\ \hline \end{array} = \frac{1}{3} \cdot \begin{array}{|c|} \hline 1 \\ \hline 1 \\ \hline 1 \\ \hline \end{array} \cdot \frac{1}{3} \cdot \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline \end{array}$$

Apply SVD to the kernel matrix:

$$\begin{aligned} \mathbf{A} &= \left[\begin{array}{c|c|c} \mathbf{u}_0 & \cdots & \mathbf{u}_{p-1} \end{array} \right] \left[\begin{array}{ccc} \sigma_0 & & \\ & \ddots & \\ & & \sigma_{p-1} \end{array} \right] \left[\begin{array}{c} \mathbf{v}_0^T \\ \cdots \\ \mathbf{v}_{p-1}^T \end{array} \right] \\ &= \sum_{j=0}^{p-1} \sigma_j \mathbf{u}_j \mathbf{v}_j^T, \end{aligned}$$

If all σ_i are 0 (apart from σ_0) then it is separable.

Example of separable filters



Roadmap: Basics of Digital Image Processing

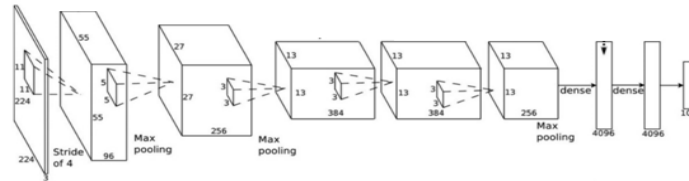
- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

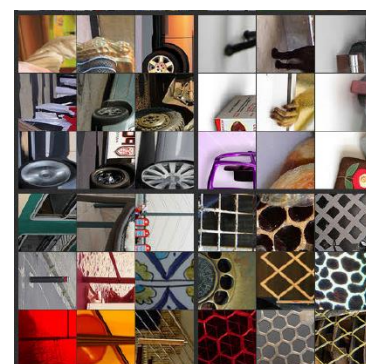
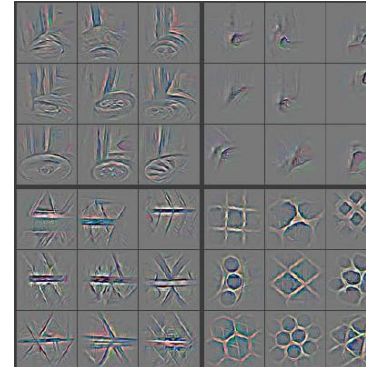
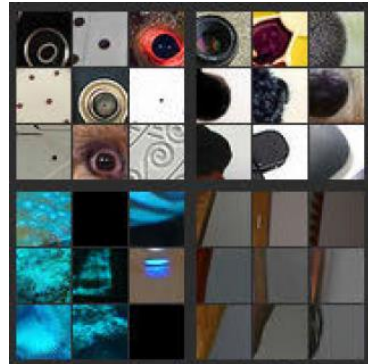
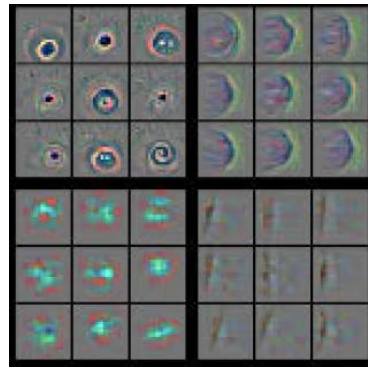
Non-linear filters

- There are many different non-linear filters
- We look at the following selection:
 - Median filter
 - Bilateral filter
 - Morphological operations
- Learning to add objects in an image via global and local filtering

Motivation: Neural networks



which image class?



Layer 1

Layer 2

Layer 3

Layer 4

Many linear filters, interleaved with non-linear operations, give rise to complex non-linear filters

[Zeiler et al. 2014]

Illustration: Salt and Pepper Noise



Original



Noised



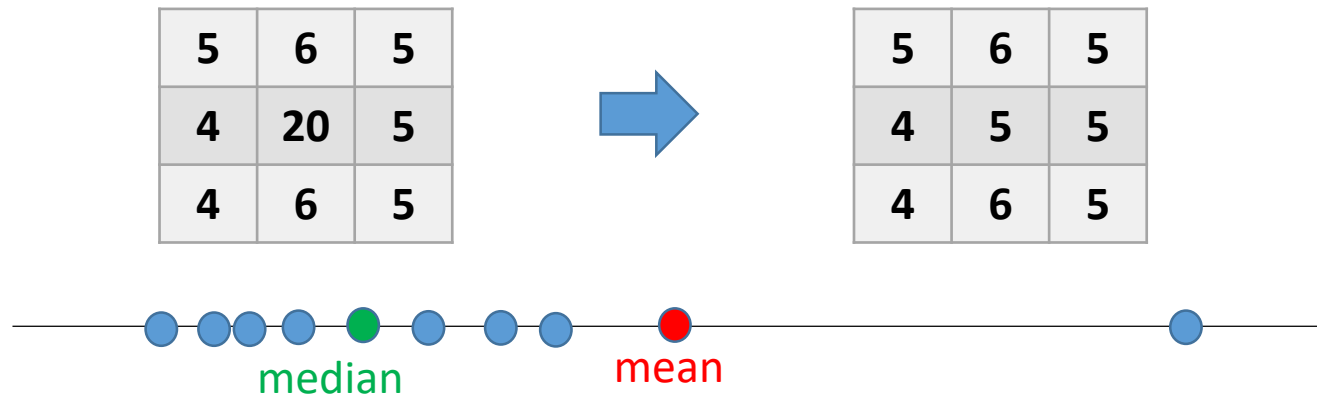
Mean



Median

Median Filter

Replace each pixel with the median in a neighbourhood:



Median filter: order the values and take the **middle** one

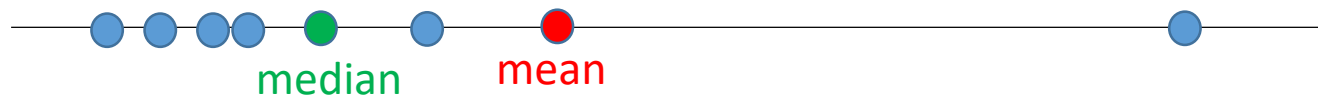
- No strong smoothing effect since values are not averaged
- Very good to remove outliers
- Used for post processing of outputs in many applications (e.g. optical flow)

Median Filter Derivation

Reminder: for Gaussian noise we did solve the following Maximum Likelihood problem:

$$y_r^* = \underset{y_r}{\operatorname{argmax}} \prod_{r' \in W(r)} \underbrace{\exp\left[-\frac{(x_{r'} - y_r)^2}{2\sigma^2}\right]}_{p(x|y)} = \underset{y_r}{\operatorname{argmin}} \sum_{r' \in W(r)} (x_{r'} - y_r)^2 = 1/|W| \sum_{r' \in W(r)} x_{r'}$$

y_r is constant in window $W(r)$



For Median we solve the following problem:

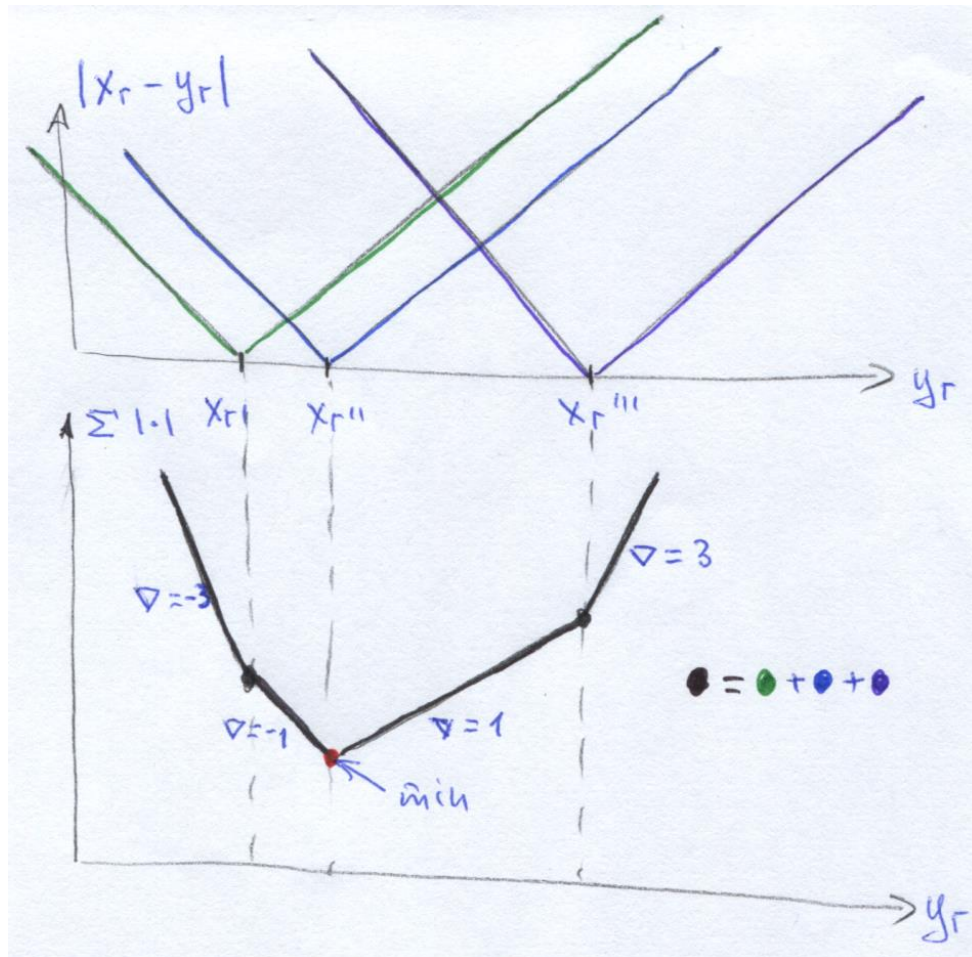
$$y_r^* = \underset{y_r}{\operatorname{argmax}} \prod_{r' \in W(r)} \underbrace{\exp\left[-\frac{|x_{r'} - y_r|}{2\sigma^2}\right]}_{p(x|y)} = \underset{y_r}{\operatorname{argmin}} \sum_{r' \in W(r)} |x_{r'} - y_r| \stackrel{!}{=} \operatorname{Median}(W(r))$$

y_r is constant in window $W(r)$ $p(x|y)$

(Due to absolute norm it is more robust)

Median Filter Derivation

Median Filter Derivation



Optimal solution is the
median of all values

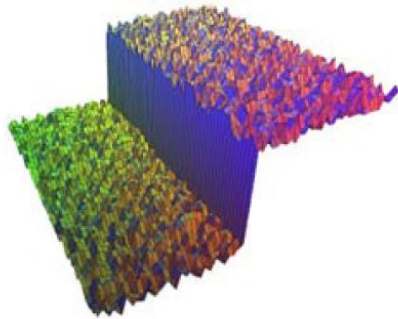
minimize the following:

$$F(y_r) = \sum_{r' \in W(r)} |x_{r'} - y_r|$$

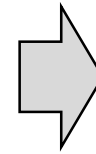
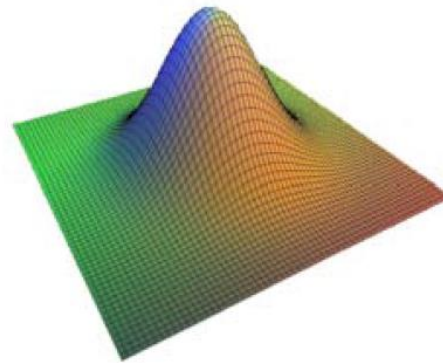
Problem: not differentiable ☹,
good news: it is convex ☺

Bilateral Filter: Motivation

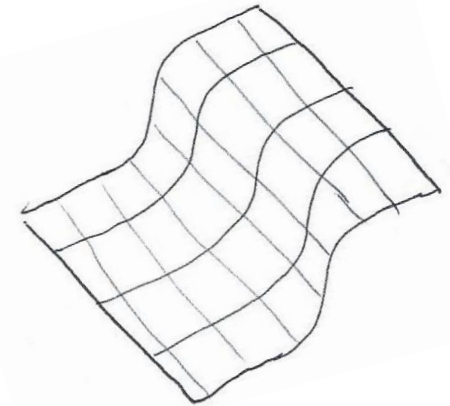
Noisy input



Gaussian Filter weights



Output (sketched)



$$g(x, y) = \sum_{k,l} f(x - k, y - l)h(k, l)$$

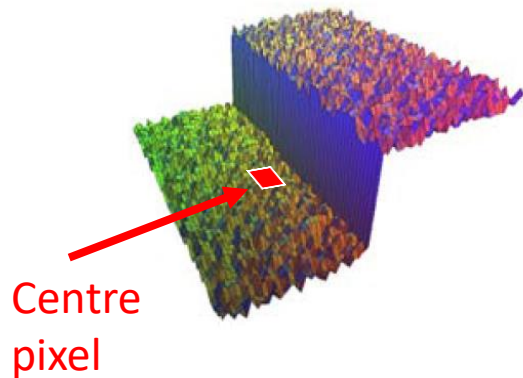
$$h(k, l) = \frac{1}{256}$$

1	4	6	4	1
4	16	24	16	4
6	24	36	24	6
4	16	24	16	4
1	4	6	4	1

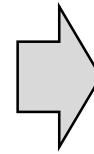
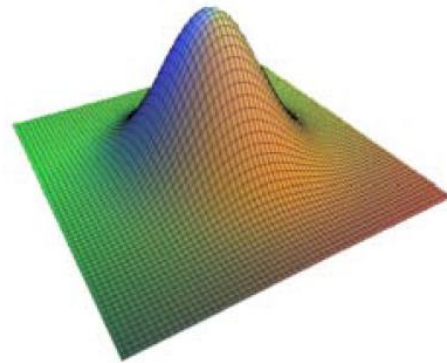
How can you make the kernel such that noise is reduced and edges are kept?

Bilateral Filter: Motivation

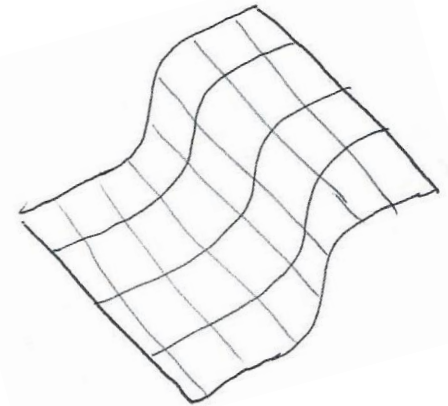
Noisy input



Gaussian Filter weights



Output (sketched)



$$g(x, y) = \sum_{k,l} f(x - k, y - l) h(k, l)$$

$$h(k, l) = \frac{1}{256}$$

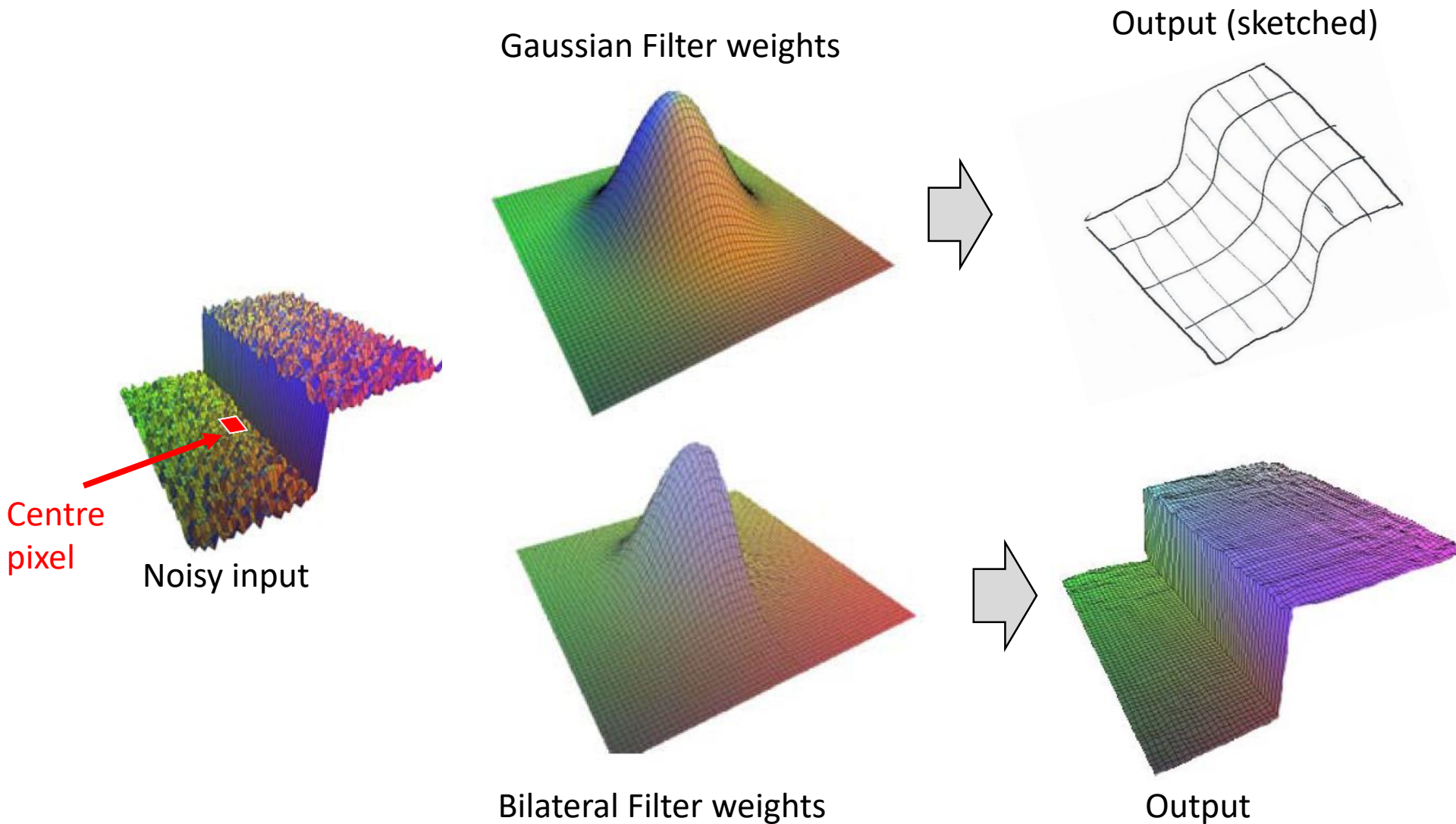
1	4	6	4	1
4	16	24	16	4
6	24	36	24	6
4	16	24	16	4
1	4	6	4	1

Reduce weights

How can you make the kernel such that noise is reduced and edges are kept?

Make the kernel position-dependent: $h_{x,y}(k, l, f(x), f(y))$

Bilateral Filter – in pictures



Bilateral Filter



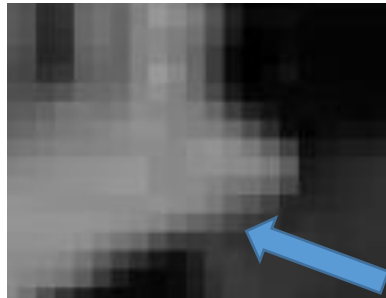
Original + Gaussian noise



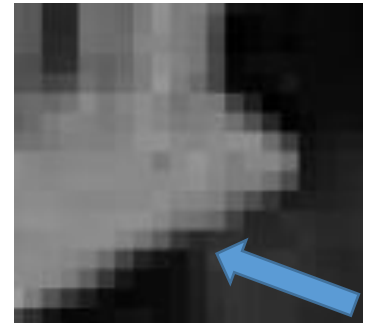
Gaussian filtered



Bilateral filtered



Edge over-smoothed

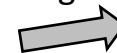


Edge not over-smoothed

Smooths
texture and
edges



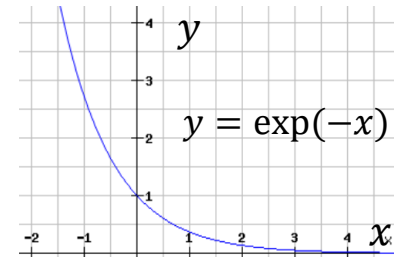
Smooths
texture and
keeps edges



Bilateral Filter – in equations

Filters looks at: a) distance to surrounding pixels (as Gaussian)
b) Intensity of surrounding pixels

$$g(i, j) = \frac{\sum_{k,l} f(k, l) w(i, j, k, l)}{\sum_{k,l} w(i, j, k, l)} \quad \text{weighted linear combination}$$



$$w(i, j, k, l) = \exp \left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_d^2} - \frac{\|f(i, j) - f(k, l)\|^2}{2\sigma_r^2} \right)$$

Same as Gaussian filter *Consider intensity*

Problem: computation is slow $O(Nw)$. Approximations can be done in $O(N)$

See a tutorial on: http://people.csail.mit.edu/sparis/bf_course/

Check: Bilateral filter is a non-linear filter

Linear Filter satisfies Additivity: $T[X + Y] = T[X] + T[Y]$

Image (X, Y) :

0	1	0
---	---	---

- 1) Operator T is (linear) boxfilter:

$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{3}$
---------------	---------------	---------------

compute filter output for central element of the image:

$$T[X + Y] = \frac{2}{3}$$

$$T[X] + T[Y] = \frac{2}{3}$$

Image (X, Y) :

.	i	.
---	---	---

$$g(i, j) = \frac{\sum_{k,l} f(k, l) w(i, j, k, l)}{\sum_{k,l} w(i, j, k, l)}$$

- 2) Operator T is a (non-linear) bilateral filter with $w(i, j, k, l) = \exp(-||f(i, j) - f(k, l)||^2)$
Compute filter output for central element of the image:

$$T[X + Y] = (0 * 0.018 + 2 * 1 + 0 * 0.018) / (0.018 + 1 + 0.018) = 2/1.036 = 1.93$$

$$\begin{aligned} T[X] + T[Y] &= 2 * [(0 * 0.36 + 1 * 1 + 0 * 0.36) / (0.36 + 1 + 0.36)] \\ &= 2 * (1/1.72) = 1.16 \end{aligned}$$

(Note: $\exp(-1) = 0.36$; $\exp(-4) = 0.018$; $\exp(0) = 1$)

Application: Bilateral Filter



Cartoonization



Original HDR

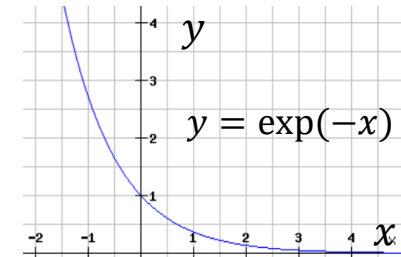


Bilateral Filter

HDR compression
(Tone mapping)

Joint Bilateral Filter

$$g(i, j) = \frac{\sum_{k,l} f(k, l) w(i, j, k, l)}{\sum_{k,l} w(i, j, k, l)}$$



$$w(i, j, k, l) = \exp \left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_d^2} - \frac{\|\tilde{f}(i, j) - \tilde{f}(k, l)\|^2}{2\sigma_r^2} \right)$$

Same as Gaussian *Consider intensity*

f is the input image, which is processed

$\tilde{f}$ is a guidance image, where we look for pixel similarity

Application: Combine Flash and No-Flash



input image f



Guidance image $\tilde{f}$



Joint Bilateral Filter

We don't care about
absolute colors

$$w(i, j, k, l) = \exp \left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_d^2} - \frac{\|\tilde{f}(i, j) - \tilde{f}(k, l)\|^2}{2\sigma_r^2} \right)$$

[Petschnigg et al. Siggraph '04]

Morphological Operations

Two steps:

1. Perform convolution with a “structural element”:
binary mask (e.g. circle or square)



2. Threshold the continuous output f to recover a binary image:

$$\theta(f, t) = \begin{cases} 1 & \text{if } f \geq t, \\ 0 & \text{else,} \end{cases}$$



original



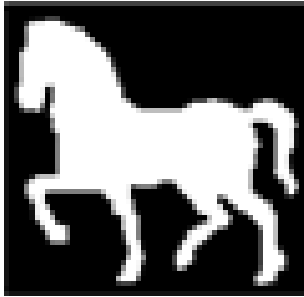
dilation f
 $\theta(f * s, 1)$



erosion f
 $\theta(f * s, S)$

of pixels
in s

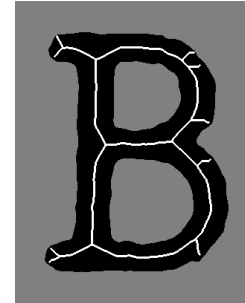
Other non-linear operations on binary images



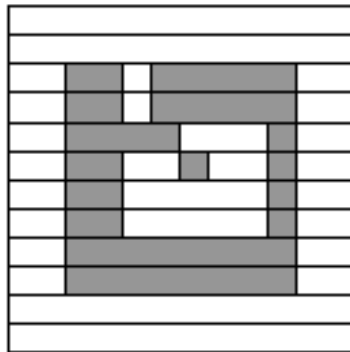
Binary
Image



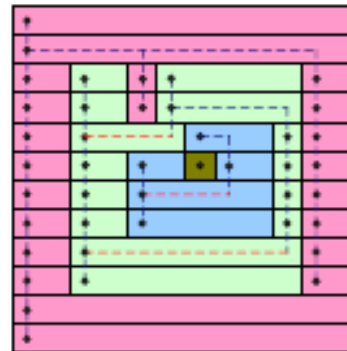
Distance
transform



Skeleton



Binary Input Image



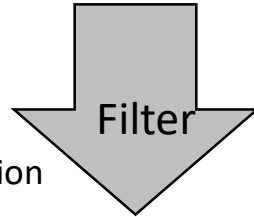
Connected components

Project: Learning Operators



before

- Global operators:
color, contrast,...
- Local operators:
Blur, noise, chromatic aberration



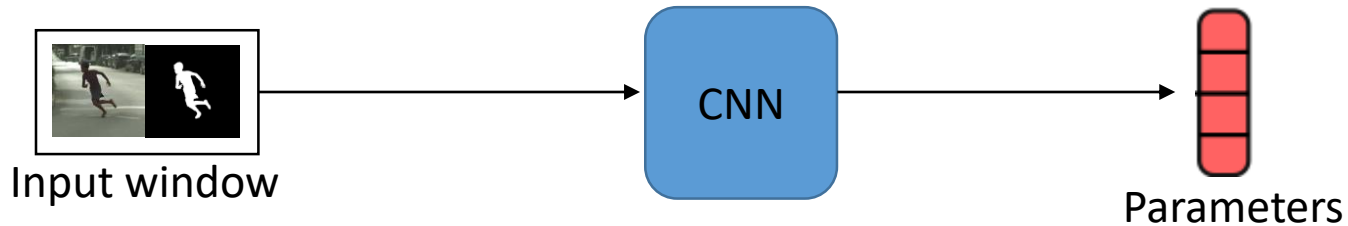
after



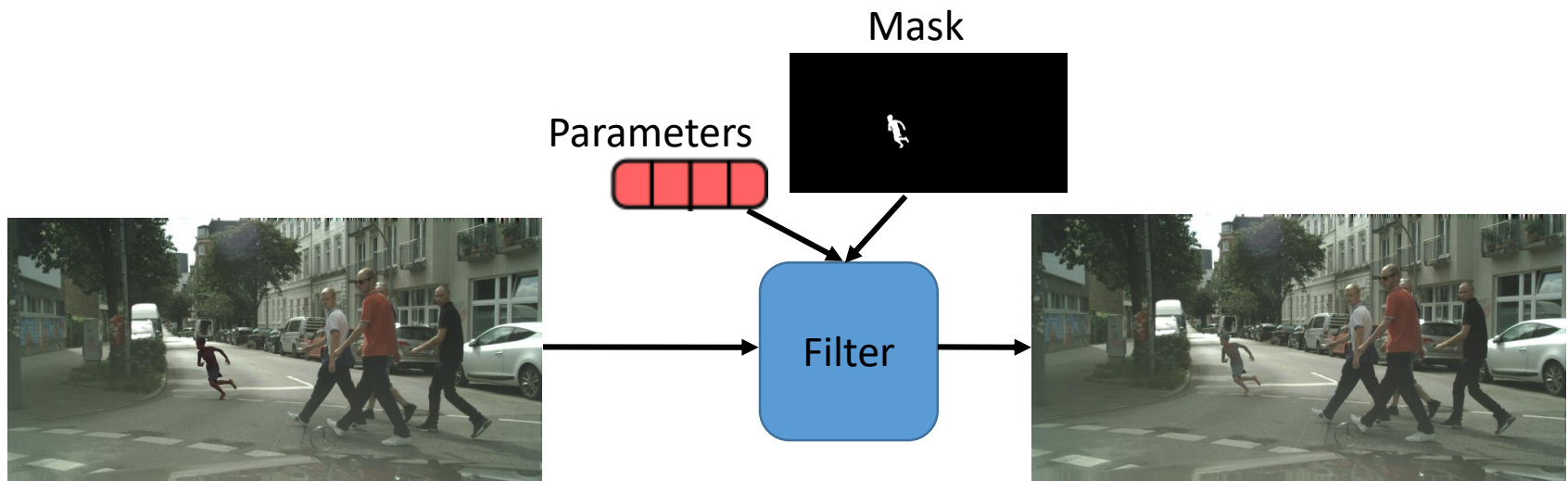
Mask

Learning Operators - Test time

1. Estimate parameters for each window of an augmented person



2. Image generation for each Window (augmented object)

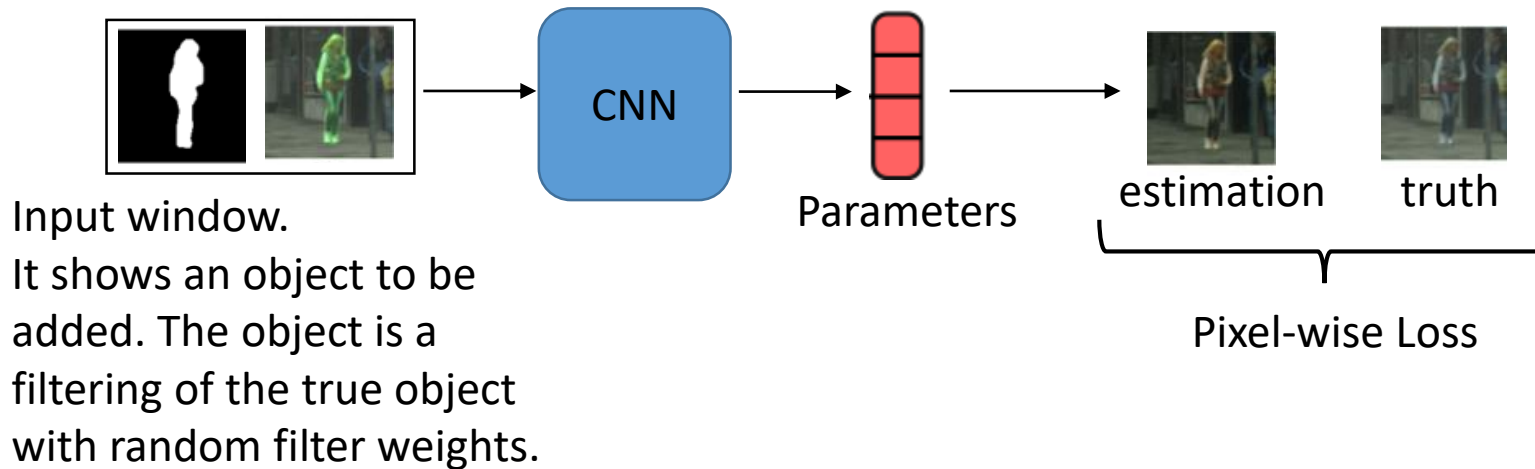


Learning Operators – Training time

1. Extract windows showing real pedestrians to train a CNN

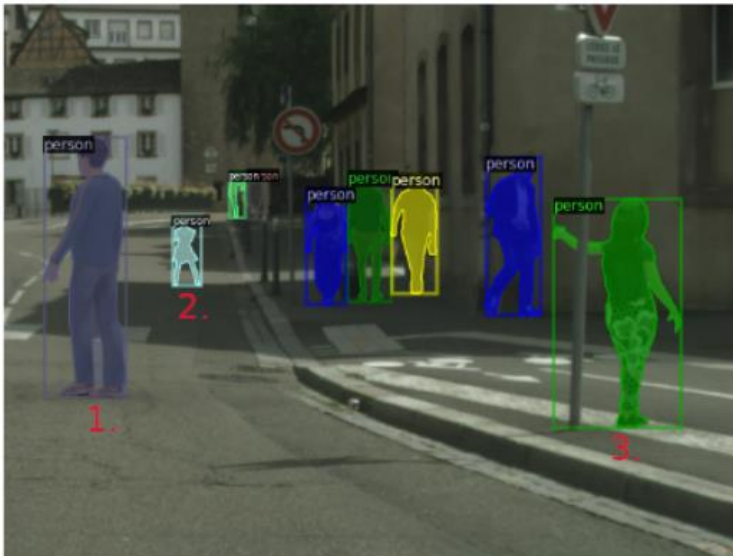


2. Training procedure Tries to find the “inverse filter”



Results

- Mask R-CNN (network for instance segmentation) is trained on augmented images containing no real pedestrians
- Synthetic People are taken from MakeHuman Software
- Evaluation on the Cityscapes validation set with real pedestrians



Instance segmentation

Average Precision on the real validation set:

- Before colour-adjustment: 4.2
- After colour-adjustment: 6.6
(higher number is better)



Before
colour-adjustment



After
colour-adjustment

Roadmap: Basics of Digital Image Processing

- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

Photography Basics

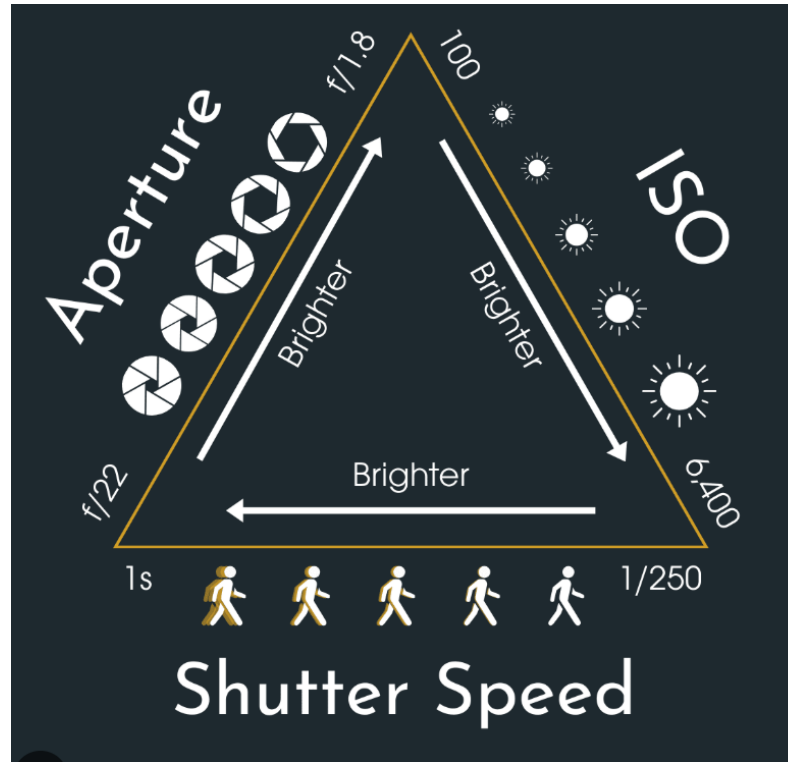


Daytime -
easy



Nighttime –
challenging

Photography Basics



The exposure triangle



Noise with high Iso setting

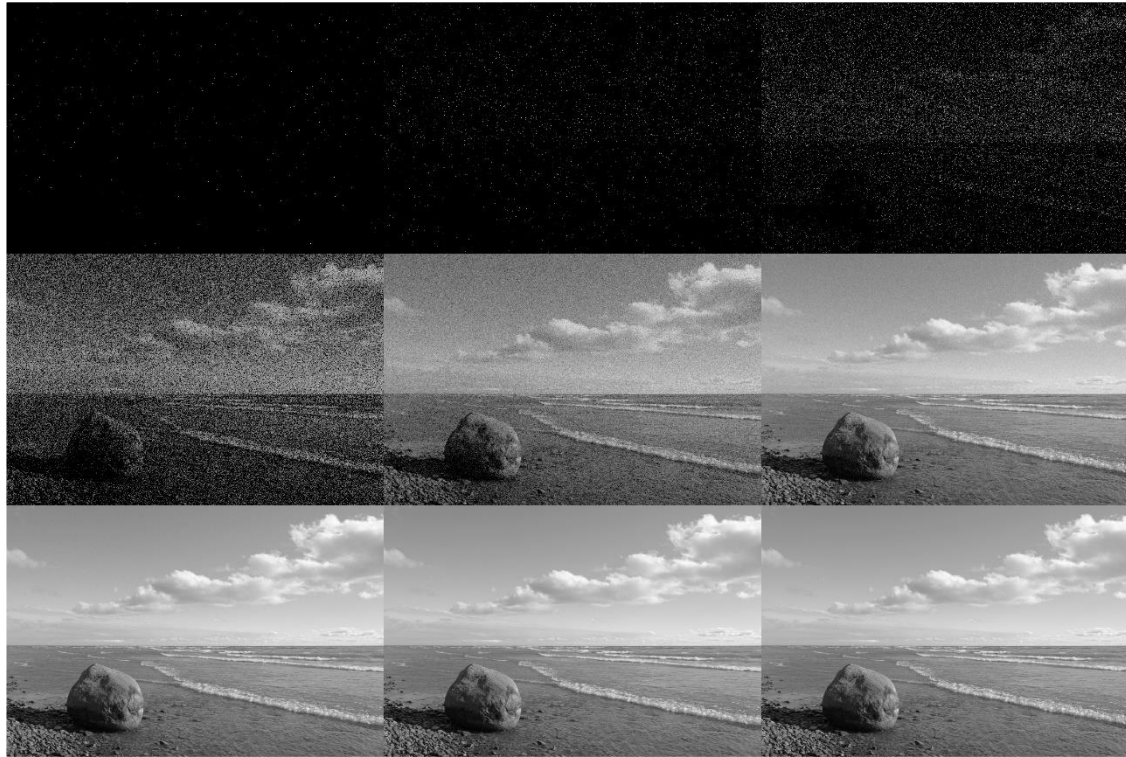


High Iso is higher Signal-to-Noise Ration

Sources of Noise

- **Shot noise (Photon noise):** This is a fundamental limitation caused by the random nature of photons as they arrive at the sensor during exposure. Even in good light, there is a small degree of randomness, but it becomes very noticeable in low-light situations.
- **Digital noise (Electronic noise):** This is generated by the camera's electronics and sensor itself. It can be a consistent, patterned noise or random, and it increases with **higher ISO**, higher temperatures, and longer exposure times.
- **Fixed-Pattern noise:** A specific type of electronic noise that is a consistent pattern across the image and becomes more apparent with high temperatures and long exposures.
- **Quantization noise:** A uniform noise that can occur during the scanning process, when the continuous analog values are converted into discrete digital values.

Shot Noise

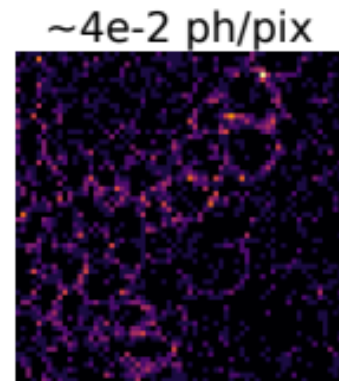
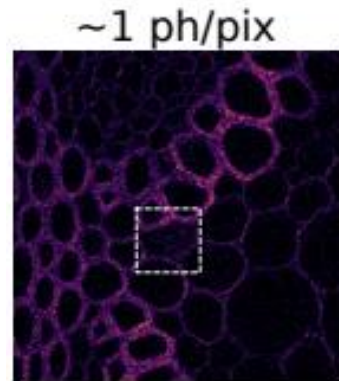


Dark pixel

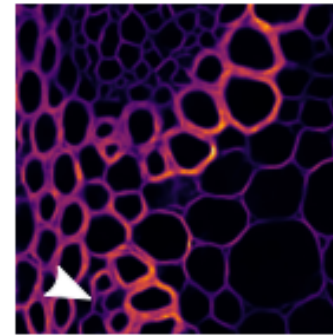
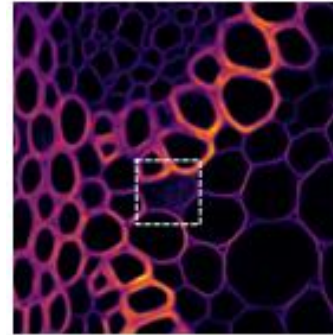
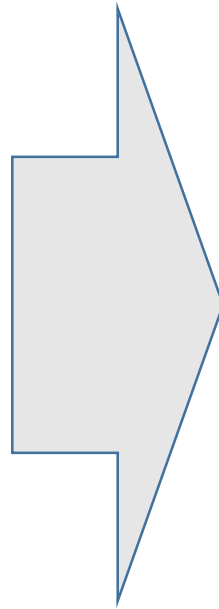


Bright Pixel

Microscopy: Increase Exposure



Under-exposed Image



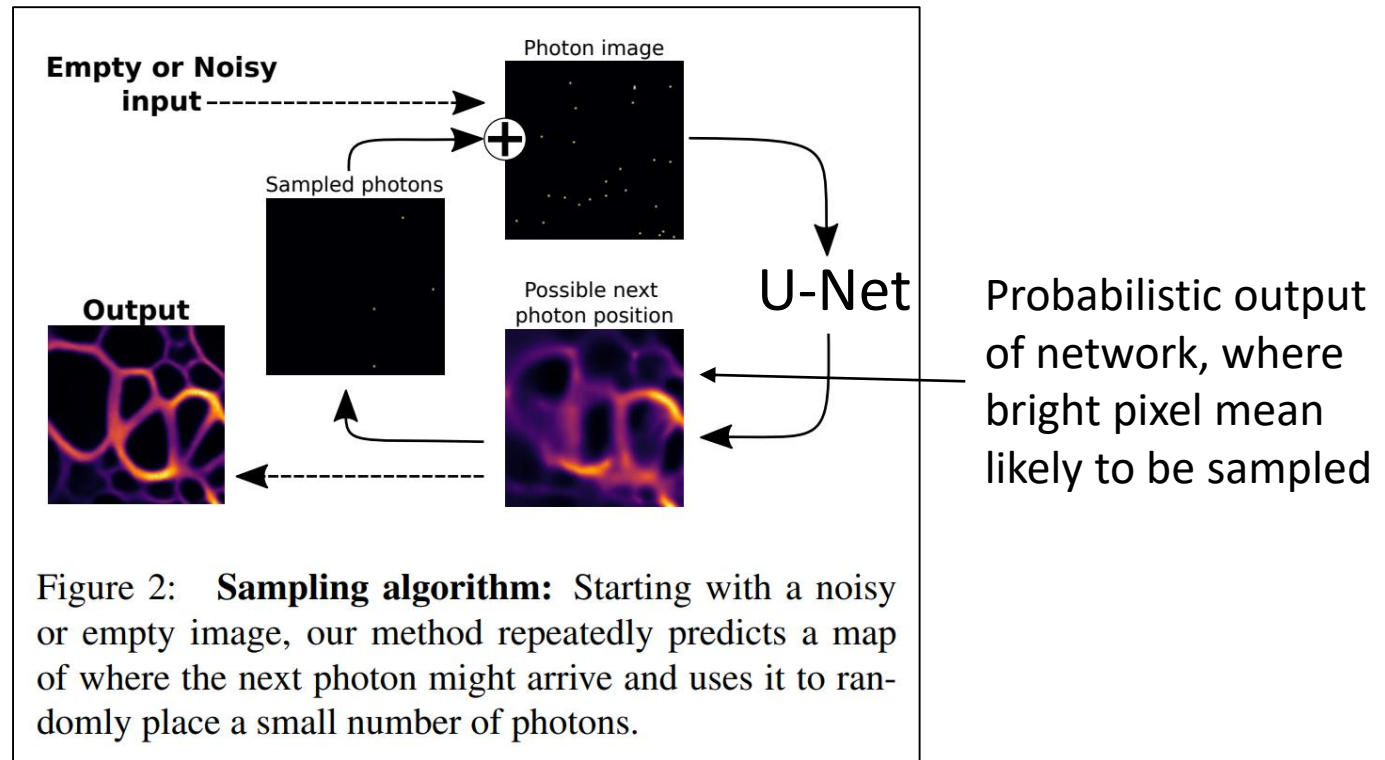
Fully-exposed Image

- **Supervised:**
Train U-Net
- **Unsupervised:**
GAP framework.
Gives also N possible outputs

GAP: [Image Denoising and the Generative Accumulation of Photons; A. Krull, et al. Arxiv; 7/2023]

Microscopy: Increase Exposure

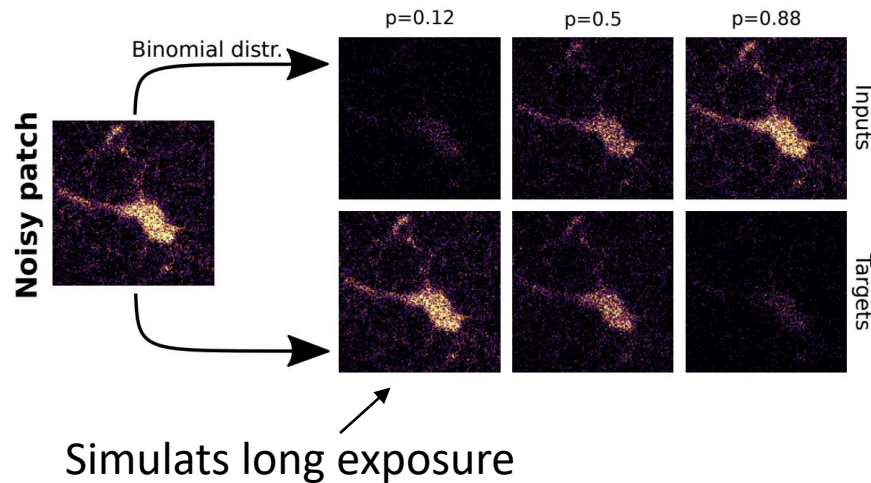
Test Time



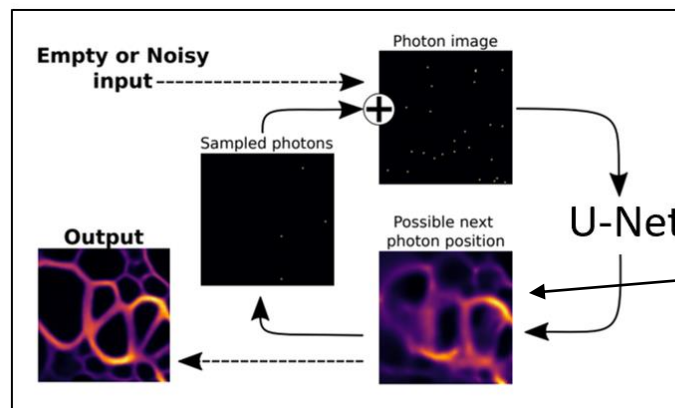
By running N times you get N different answers

Microscopy: Increase Exposure

Generate Training Data automatically:



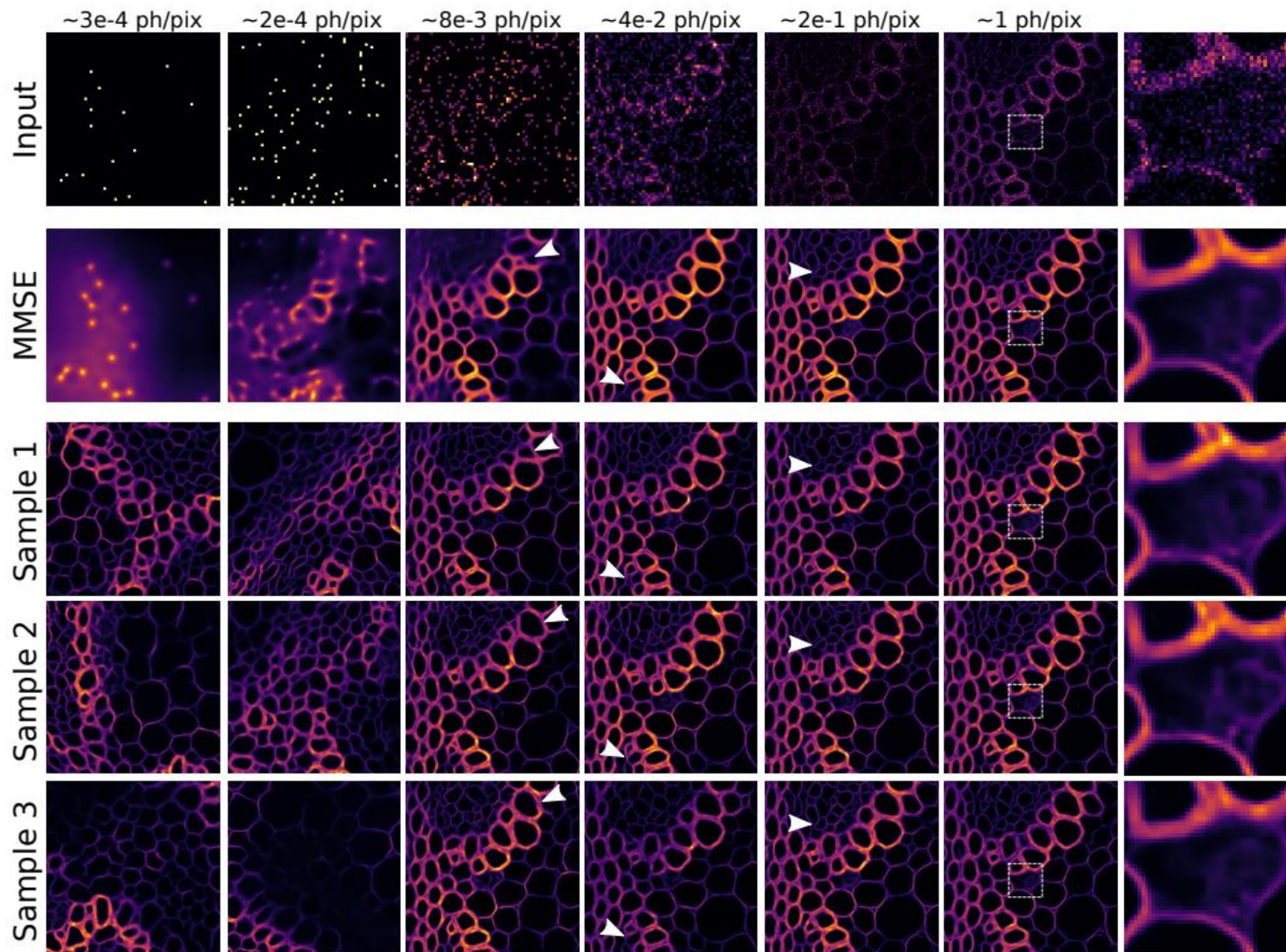
- Simulating Photon arrivals
- Train the U-Net to predict the target (i.e. where next photon is sampled)



Probabilistic output of network, where bright pixel mean likely to be sampled

GAP: [Image Denoising and the Generative Accumulation of Photons; A. Krull, et al. Arxiv; 7/2023]

Microscopy: Increase Exposure



GAP: [Image Denoising and the Generative Accumulation of Photons; A. Krull, et al. Arxiv; 7/2023]

Roadmap: Basics of Digital Image Processing

- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

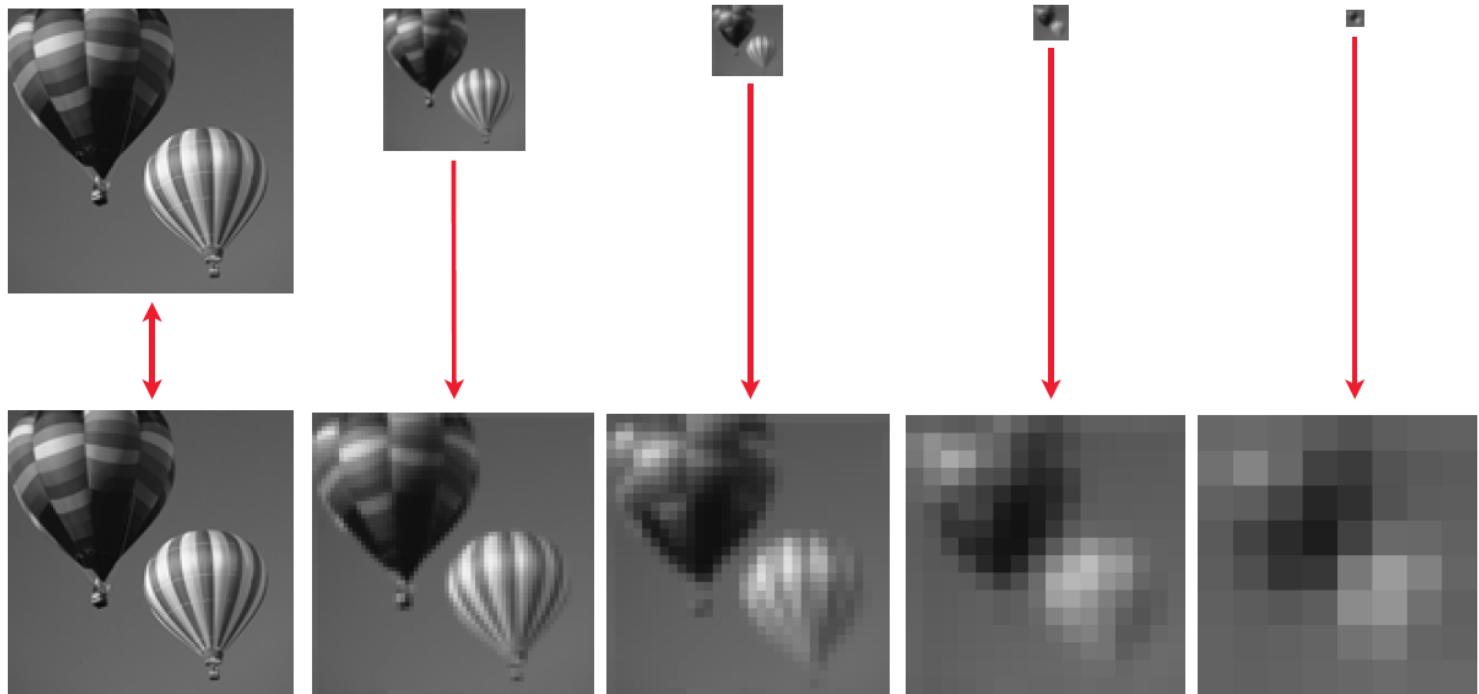
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

Gaussian Image Pyramid

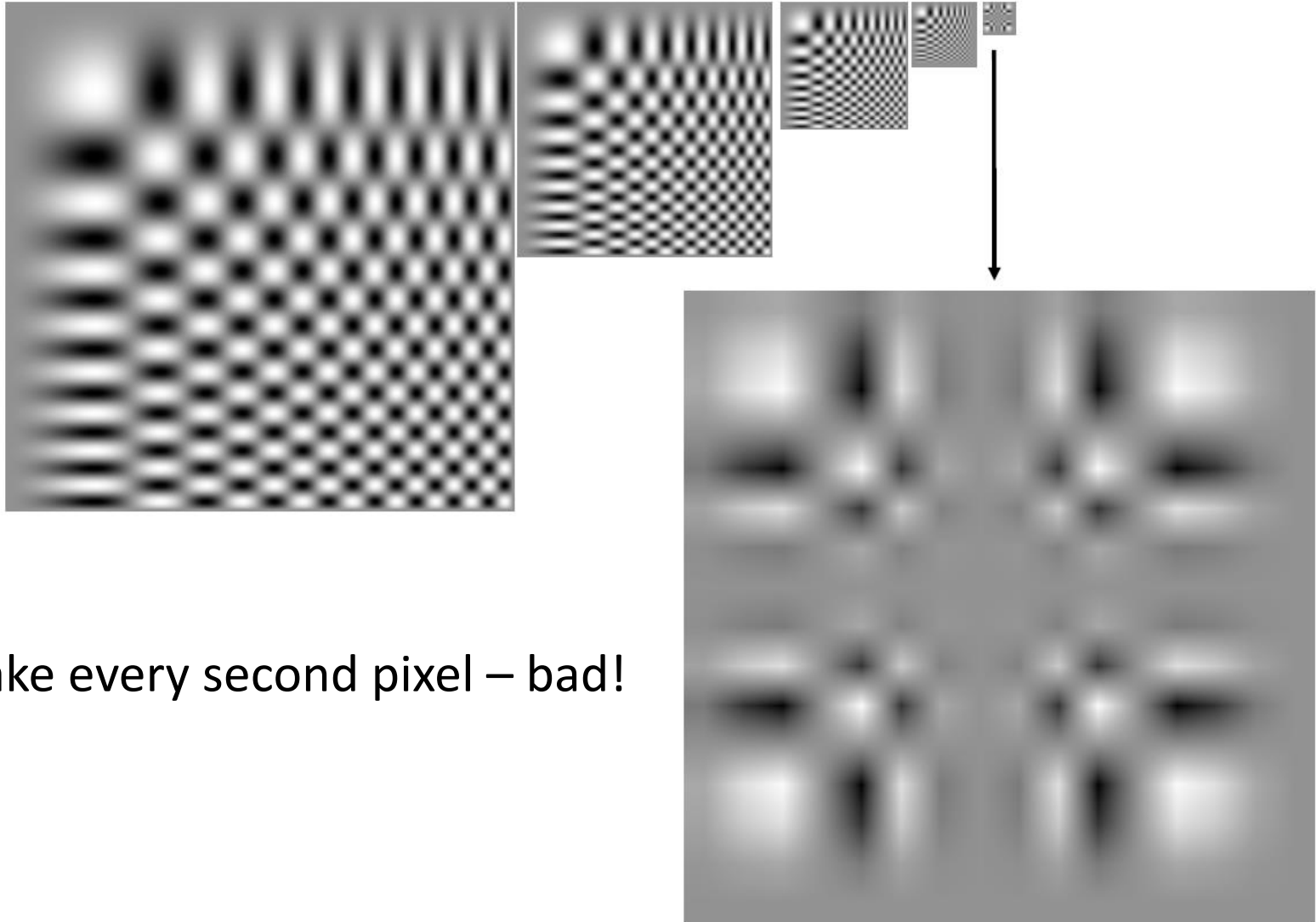
- Represent Image at multiple resolution

High resolution

Low resolution



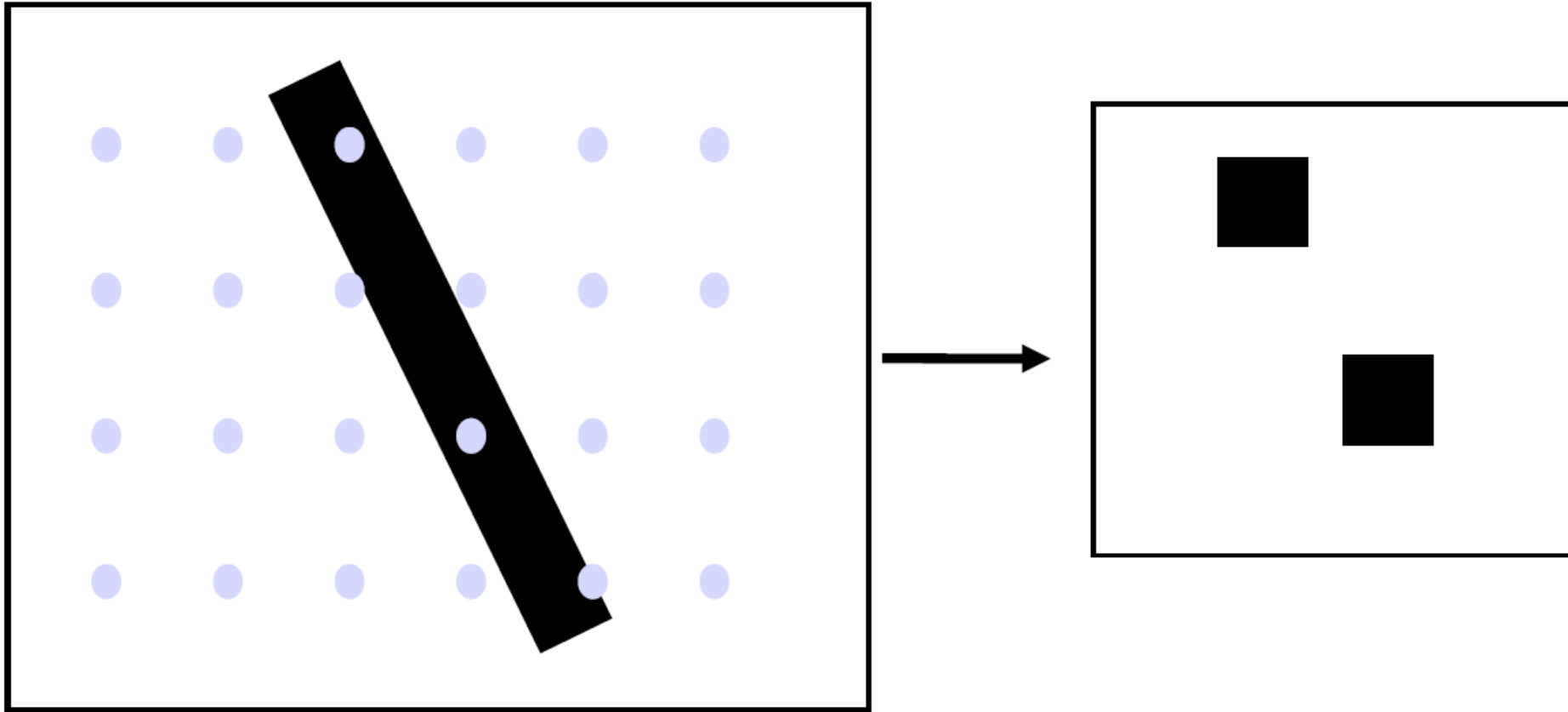
A naive approach



Take every second pixel – bad!

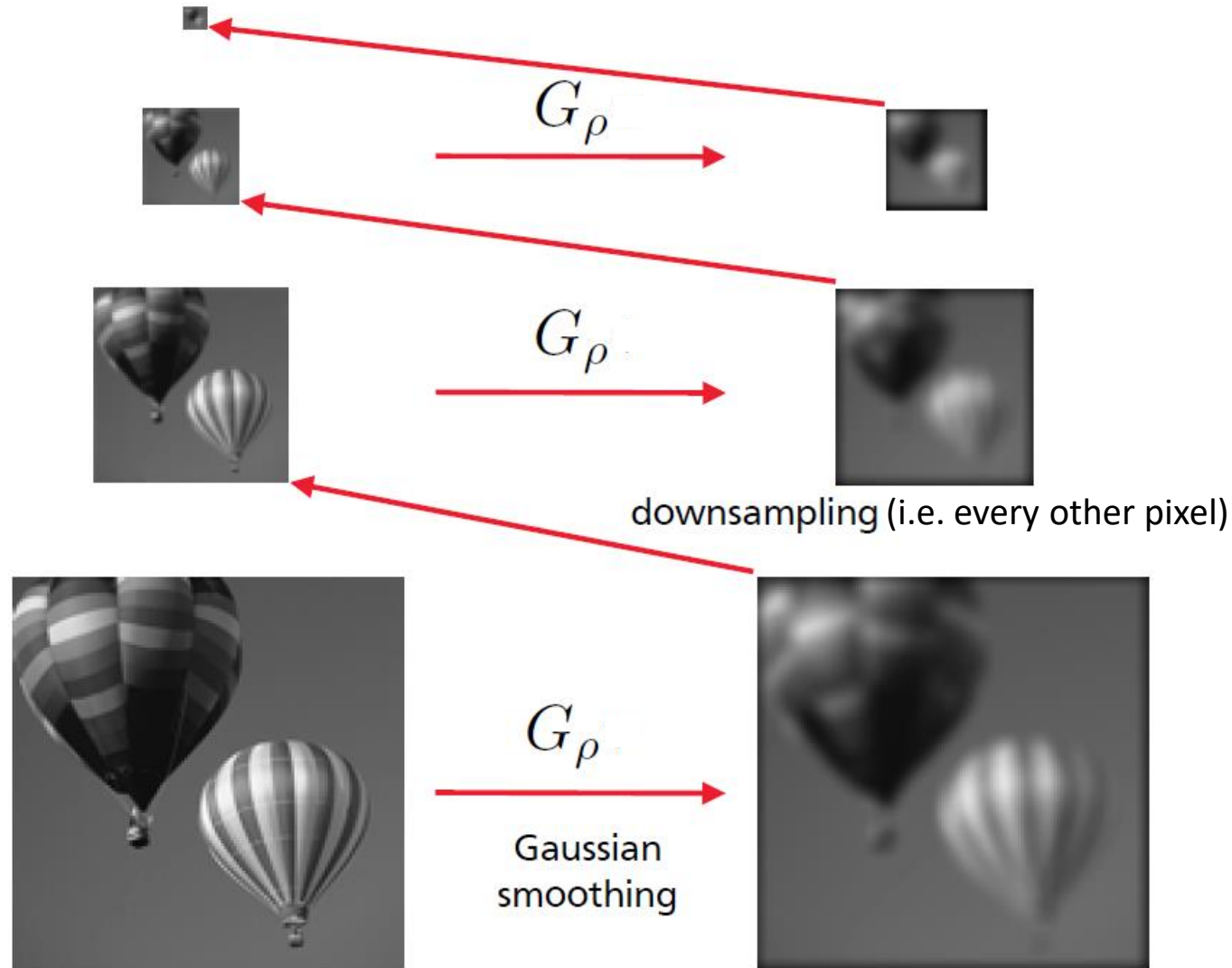
[From book: Computer Vision A modern Approach, Ponce and Forsyth]

Problem: Aliasing Effects



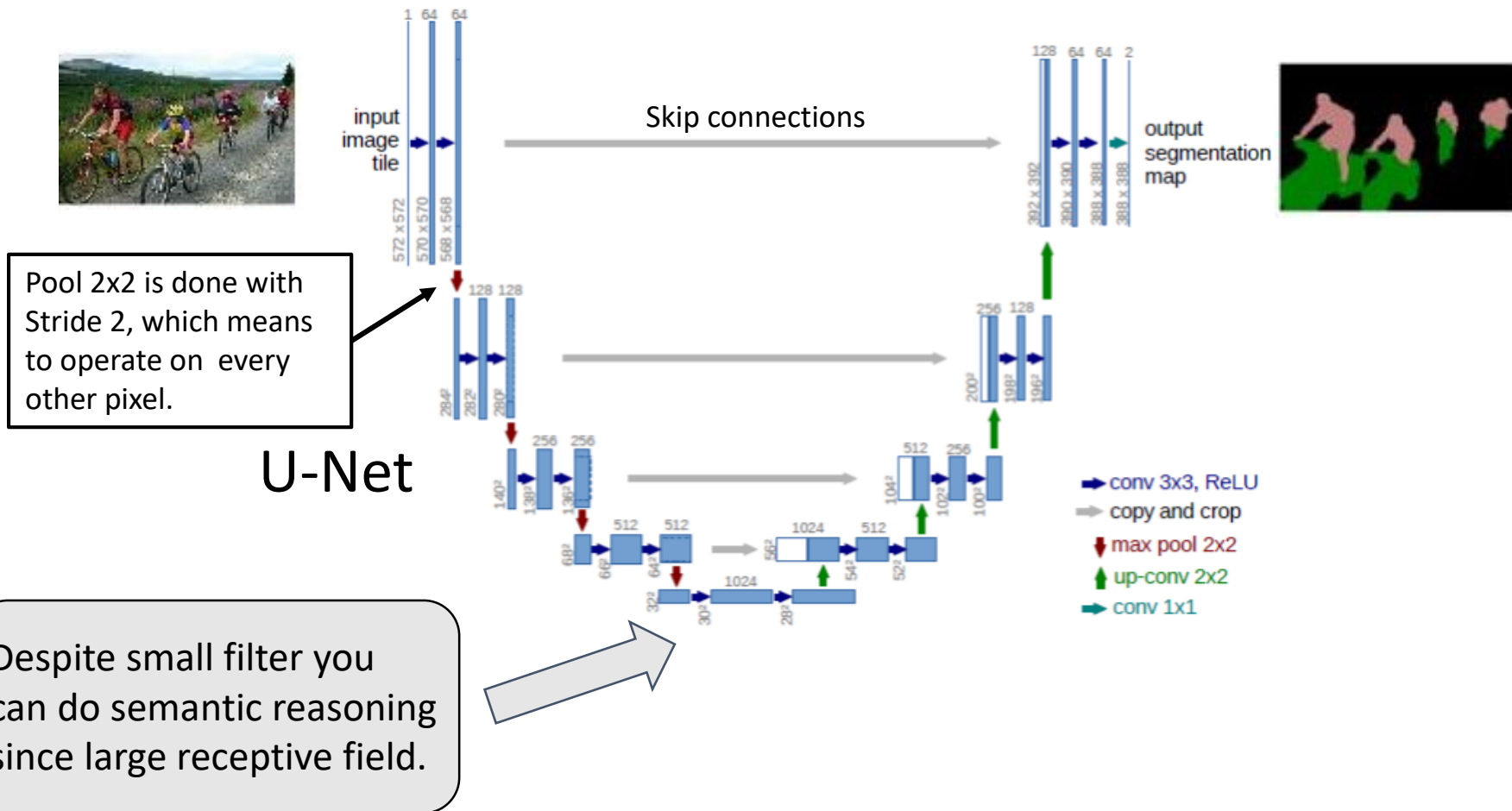
Problem: High frequencies (sharp transitions) are lost

Solution: Smooth before downsampling



Connection to Neural Networks

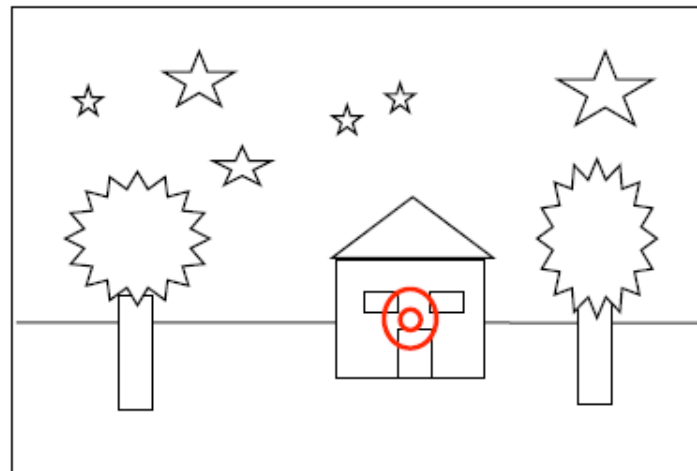
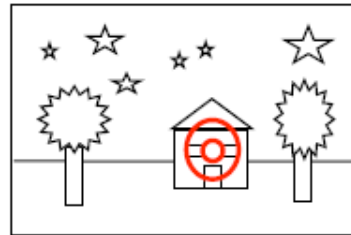
Neural networks may/should do internally the right thing



Application: Template search



Search template:



[from Irani, Basri]

Application: Segmenting large images



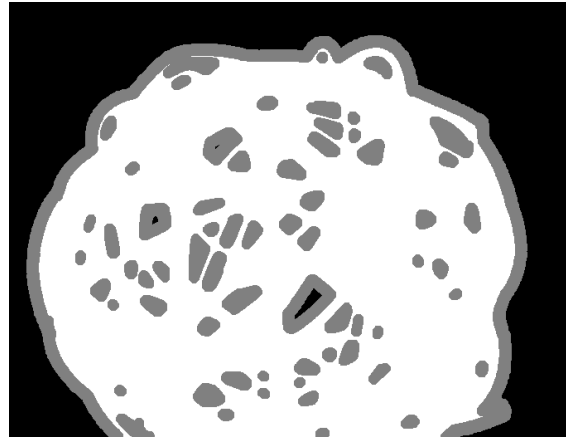
Small image
(100x100)



Small segmentation
result



Large image, e.g. 10 MPixel



Trimap: created
from small image



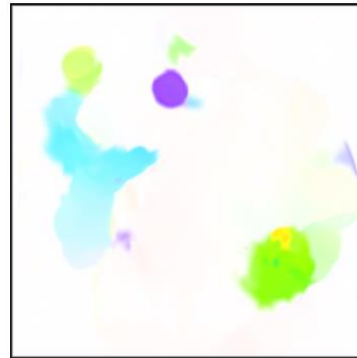
Segmentation
large image

“Banded Segmentation”

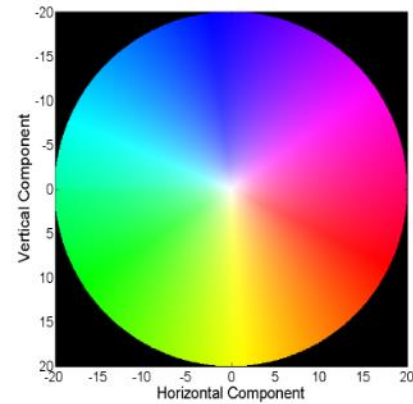
Application: Large Label Space



2 images
(overlaid)



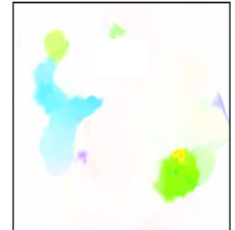
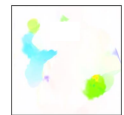
Motion
200 × 200 possible
discrete movements
(40.000 labels)



Color coding

Approach:

1. solve problem on small image (x5 downscale)
with 40 x 40 label space (**coarse motion**)
(1600 labels)
2. Do on full resolution only in 5x5 neighbourhood
around each solution (**add fine motion**)
(25 labels)



(problem: small, moving
objects can get lost)